Taller Práctico de Arquitectura de Software: EventFlow y Auditoría Integral del Caso SUMP con SAAM, ATAM, Heurísticas de Nielsen, DevOps e Infraestructura como Código

Hanna Katherine Abril Gongora

Programa de Ingeniería de Software, Universidad Manuela Beltrán

[ASIGNATURA]

[DOCENTE]

13 de octubre de 2026

Tabla de Contenido

Introducción 4

Objetivos 5

Objetivo General 5

Objetivos Específicos 5

Marco Conceptual 5

Ejercicio Práctico 1: EventFlow 7

Fundamentos 7

El Dominio y el Backlog 9

Monolito vs. Microservicios 12

DevOps, Infraestructura como Código y Deuda Técnica 18

Evaluación de la Arquitectura y de la Experiencia de Usuario 31

Ejercicio Práctico 2: Auditoría Integral del Caso SUMP 40

Módulo A: Aplicación de ATAM 41

Módulo B: Aplicación de SAAM 45

Módulo C: Auditoría UX con las Heurísticas de Nielsen 46

Cuestionario del Taller 48

Pregunta 1. Rendimiento (Latencia) 48

Pregunta 2. Escalabilidad (Horizontal) 48

Pregunta 3. Fiabilidad (Tolerancia a Fallos) 49

Pregunta 4. Seguridad (Autorización) 50

Pregunta 5. Seguridad (Integridad de Datos) 50

Pregunta 6. Usabilidad (Eficiencia) 51

Pregunta 7. Mantenibilidad (Modularidad) 51

Pregunta 8. Rendimiento (Concurrencia) 51

Pregunta 9. Disponibilidad (99,99 %) 52

Pregunta 10. Trade-off (Rendimiento vs. Consistencia) 52

Pregunta 11. Stakeholders Clave para la Sesión ATAM 53

Pregunta 12. Atributos con Mayor Prioridad en el Árbol de Utilidad 53

Pregunta 13. Escenario de Calidad de Rendimiento (Estímulo–Respuesta–Medida) 54

Pregunta 14. Mapeo de la Decisión de Microservicios al Escenario de Escalabilidad 54

Pregunta 15. Riesgo de Bloquear el Asiento en la Base de Datos al Seleccionarlo 55

Pregunta 16. Trade-off del Cifrado de Extremo a Extremo 55

Pregunta 17. Punto de Sensibilidad Crítico para la Mantenibilidad 56

Pregunta 18. Preguntas al Arquitecto sobre la Caída de la Pasarela de Pagos 56

Pregunta 19. Hallazgo Más Valioso para el Gerente de Proyecto 57

Conclusiones 58

Referencias 59

Introducción

La arquitectura de software se define como “la estructura, comprendida por los elementos de software, las propiedades visibles de esos elementos y las relaciones entre ellos” (Bass et al., 2021). Esta definición tiene una consecuencia práctica que guía todo este informe: las decisiones arquitectónicas no se juzgan por su elegancia, sino por la forma en que permiten (o impiden) que el sistema cumpla sus atributos de calidad —escalabilidad, rendimiento, seguridad, disponibilidad, mantenibilidad y usabilidad— frente a los objetivos del negocio.

El taller se desarrolla en dos ejercicios. En el primero diseño **EventFlow**, un sistema simple de venta de entradas a eventos: identifico requisitos no funcionales (NFR) y stakeholders, construyo el backlog en Trello y el modelo de dominio siguiendo Domain-Driven Design (Evans, 2003), diseño la solución dos veces (monolito en capas y microservicios orientados a eventos), simulo un ciclo DevOps con GitHub, GitHub Actions, SonarCloud y Terraform sobre los tres archivos entregados en el enunciado (`index.html`, `.github/workflows/ci.yml` y `main.tf`), y audito el diseño con las heurísticas de Nielsen (1994a), ATAM (Kazman et al., 2000) y SAAM (Kazman et al., 1994).

En el segundo ejercicio actúo como consultora para auditar el **SUMP** (Sistema Único de Matrículas y Pagos) de una universidad ficticia: una arquitectura “Frankenstein” —monolito Java on-premise conectado por una VPN inestable a un microservicio Node.js en AWS— con una interfaz diseñada sin criterios de usabilidad. Sobre ese caso aplico ATAM para exponer el conflicto entre disponibilidad/rendimiento y seguridad/consistencia, SAAM para medir la modificabilidad frente a un cambio legal de pasarela de pagos, y una evaluación heurística de la interfaz. Al final respondo las diecinueve preguntas del cuestionario con valores medibles.

Objetivos

Objetivo General

Aplicar los conceptos de atributos de calidad, diseño guiado por el dominio, estilos arquitectónicos, DevOps, infraestructura como código y evaluación de arquitecturas y de usabilidad (SAAM, ATAM y heurísticas de Nielsen) en el diseño de EventFlow y en la auditoría del caso SUMP, argumentando cada decisión como ingeniera de software.

Objetivos Específicos

- Identificar y priorizar los NFR de EventFlow y relacionarlos con atributos de calidad y stakeholders.
- Construir un backlog ágil y un modelo de dominio que distinga el Dominio Core de los subdominios genéricos.
- Comparar un monolito en capas con una arquitectura de microservicios orientada a eventos frente al NFR de 10.000 usuarios por minuto.
- Simular un ciclo DevOps completo (código, análisis de deuda técnica, integración continua e IaC) con los archivos del taller.
- Evaluar la usabilidad de un wireframe con las heurísticas de Nielsen y proponer su corrección.
- Aplicar ATAM y SAAM a EventFlow y al SUMP, identificando trade-offs, puntos de sensibilidad, riesgos y el impacto de escenarios de cambio.

Marco Conceptual

SAAM (Software Architecture Analysis Method) es uno de los primeros métodos del Software Engineering Institute (SEI) y se centra en la modificabilidad: los stakeholders proponen escenarios de cambio y se cuenta cuántos componentes habría que modificar; si el impacto queda

aislado, la arquitectura es modificable, y si se dispersa, es rígida (Kazman et al., 1994). Su limitación es que no analiza simultáneamente rendimiento o seguridad.

ATAM (Architecture Tradeoff Analysis Method) es su evolución: evalúa las decisiones arquitectónicas contra varios atributos de calidad a la vez usando un **árbol de utilidad**, y produce **puntos de sensibilidad** (decisiones que afectan fuertemente un atributo), **puntos de trade-off** (decisiones que mejoran un atributo y empeoran otro), **riesgos** y **no-riesgos** (Kazman et al., 2000; Clements et al., 2002). ATAM no dictamina si una arquitectura es "buena", sino si es adecuada para los objetivos de negocio dados los sacrificios que se aceptan.

Las **heurísticas de Nielsen** son diez principios generales de diseño de interacción publicados en 1994 que permiten evaluar la usabilidad de una interfaz sin pruebas costosas con usuarios (Nielsen, 1994a, 1994b): visibilidad del estado del sistema; relación entre el sistema y el mundo real; control y libertad del usuario; consistencia y estándares; prevención de errores; reconocer antes que recordar; flexibilidad y eficiencia de uso; diseño estético y minimalista; ayudar a reconocer, diagnosticar y recuperarse de errores; y ayuda y documentación.

Tabla 1

Síntesis comparativa de los métodos de evaluación

Método	¿Qué evalúa?	¿Qué busca encontrar?	Usuarios principales
SAAM	Modificabilidad interna	Cuánto cuesta (en esfuerzo) cambiar un módulo	Arquitectos de software
ATAM	Múltiples atributos de calidad	Riesgos y trade-offs entre rendimiento, seguridad, etc.	Arquitectos y líderes técnicos
Nielsen	Usabilidad externa (UX/UI)	Fricciones, confusión o bloqueos en la interfaz	Diseñadores UX y desarrolladores frontend

Nota. Adaptado del enunciado del taller.

Ejercicio Práctico 1: EventFlow

EventFlow es un sistema (muy) simple de venta de entradas a eventos. Las herramientas usadas fueron Trello (gestión ágil), diagrams.net como alternativa a Miro (modelado), Figma (UX), GitHub y GitHub Actions (código y CI/CD), SonarCloud (deuda técnica), Google Cloud Shell con Terraform (IaC) y Swagger Hub (diseño de API). Todos los artefactos están en el repositorio `taller-arquitectura`: https://github.com/hannakathe/taller-arquitectura.

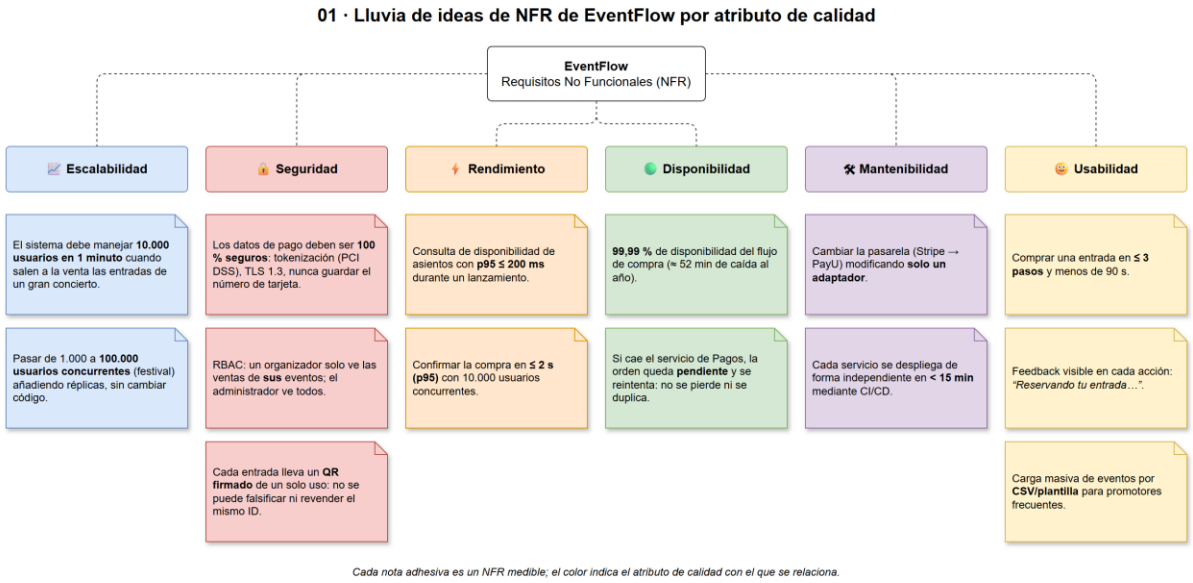
Fundamentos

Requisitos No Funcionales (Lluvia de Ideas)

Un NFR solo es útil si es **medible**: "rápido" no se puede probar, "p95 ≤ 200 ms" sí. Por eso, en la lluvia de ideas convertí cada intuición en un requisito con número y lo relacioné con su atributo de calidad mediante un código de colores (Figura 1). Los dos ejemplos del enunciado quedan así: "manejar 10.000 usuarios en 1 minuto cuando salen a la venta las entradas de un gran concierto" es un NFR de **escalabilidad** (capacidad de absorber más carga agregando recursos), y "los datos de pago deben ser 100 % seguros" es de **seguridad**, que concreto como tokenización de la tarjeta con la pasarela (cumplimiento PCI DSS) para que EventFlow nunca almacene el número de la tarjeta.

Figura 1

Lluvia de Ideas de Requisitos No Funcionales de EventFlow por Atributo de Calidad



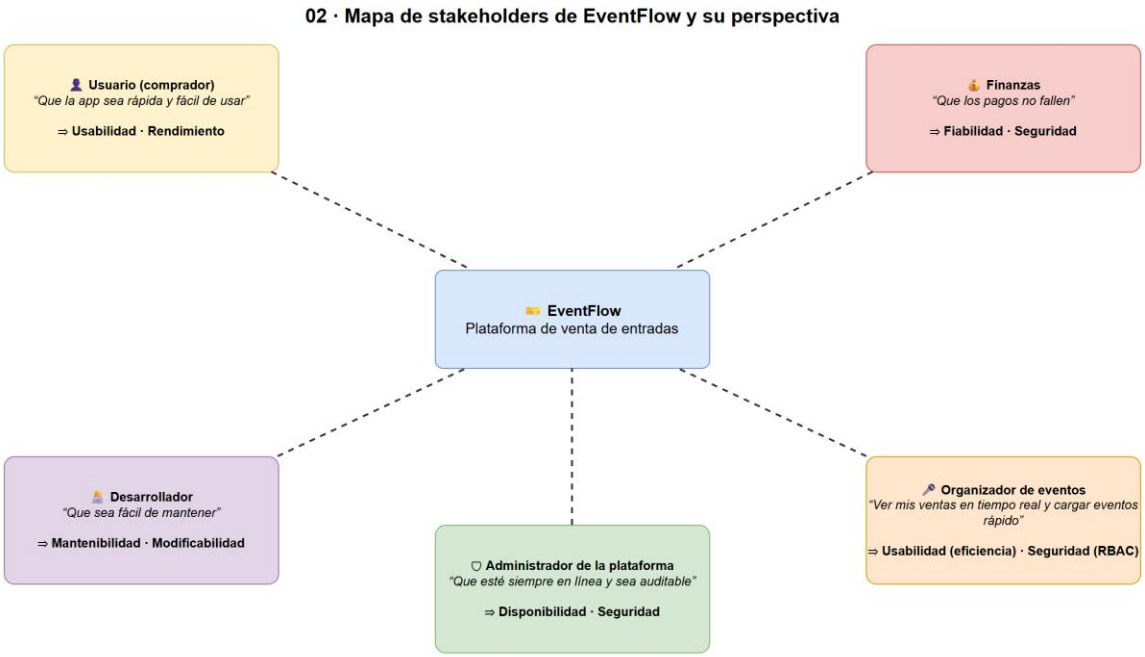
Nota. Elaboración propia en diagrams.net (archivo `diagramas/01_nfr_eventflow.drawio`). Cada color corresponde a un atributo de calidad.

Mapa de Stakeholders

Cada stakeholder mira el mismo sistema desde una perspectiva distinta, y esas perspectivas se traducen en atributos de calidad que a veces compiten (Figura 2). El usuario pide que la app sea rápida y fácil (usabilidad y rendimiento); Finanzas, que los pagos no fallen (fiabilidad y seguridad); el desarrollador, que sea fácil de mantener (mantenibilidad). Añadí al organizador de eventos, que necesita ver sus ventas y cargar eventos rápido, y al administrador, que necesita disponibilidad y auditoría. Este mapa es la entrada natural del árbol de utilidad de ATAM.

Figura 2

Mapa de Stakeholders de EventFlow y su Perspectiva



Nota. Elaboración propia en diagrams.net (`diagramas/02_stakeholders.drawio`).

El Dominio y el Backlog

Backlog en Trello

Creé el tablero "EventFlow — Backlog" con las listas *Backlog*, *Sprint 1*, *En progreso* y *Hecho*, y diez historias de usuario con el formato "Como ___, quiero ___ para ___" (Cohn, 2004). Solo las cinco historias del Sprint 1 llevan descripción y criterios de aceptación (Figuras 3 y 4); el texto completo está en `backlog_trello.txt`.

Figura 3

Tablero de Trello de EventFlow con las Listas Backlog, Sprint 1, En Progreso y Hecho

PENDIENTE: fig03_trello_tablero.png

Tablero completo con las 10 historias y las 5 del Sprint 1

Nota. Captura de pantalla de Trello.

Figura 4

Tarjeta de la Historia HU-03 con sus Criterios de Aceptación

PENDIENTE: fig04_trello_tarjeta.png

Tarjeta HU-03 abierta con descripción y checklist de criterios

Nota. Captura de pantalla de Trello.

¿Cuál es el valor de negocio de la historia "Como comprador, quiero ver la lista de eventos disponibles para poder elegir uno"? Es la puerta de entrada del embudo de ventas: si el comprador no descubre los eventos, ninguna otra funcionalidad genera ingresos. También entrega valor al organizador, porque su evento se vuelve visible, y permite medir desde el primer sprint una métrica de negocio (visitas al catálogo frente a compras, es decir, la tasa de conversión). Por eso es la primera historia del backlog: es pequeña, independiente y habilita a todas las demás.

¿Necesitamos detallar todas las historias ahora? No, solo aquellas que se desarrollarán en el primer sprint. Detallar todo por adelantado es desperdicio: los requisitos cambian a medida

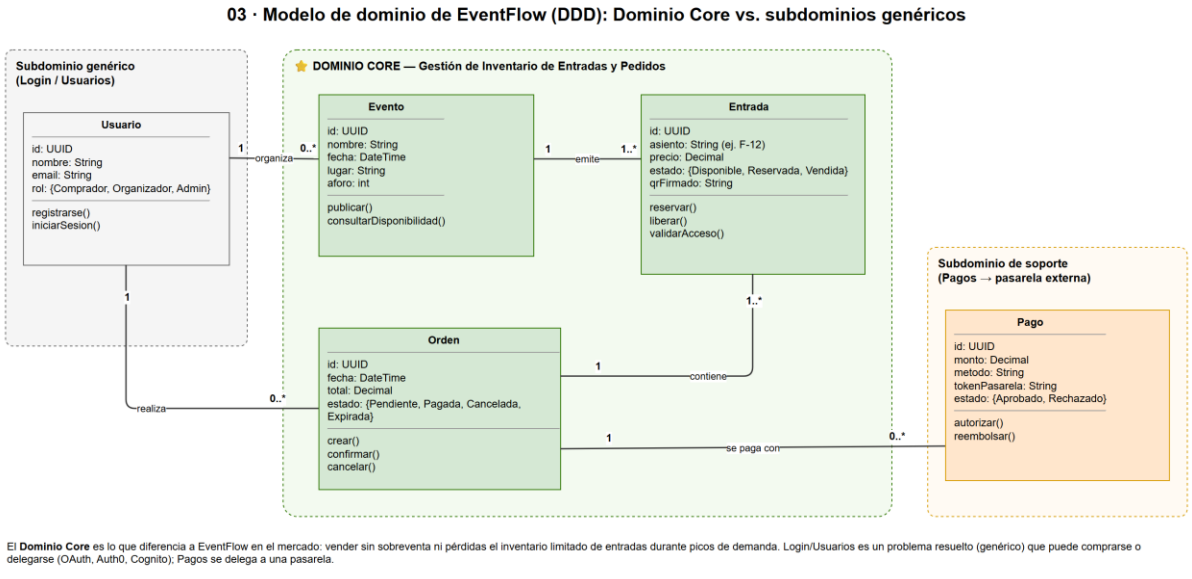
que el equipo y el cliente aprenden, y el detalle escrito hoy para la HU-10 (validación del QR en la puerta) probablemente quedará obsoleto cuando lleguemos a ella. Un backlog sano es "apropiadamente detallado": lo próximo está listo para construirse (criterios de aceptación, estimación) y lo lejano es apenas una línea que se refina en sesiones de *backlog refinement*. Así se prioriza por valor, se reduce el riesgo de construir lo equivocado y se respeta el principio ágil de responder al cambio.

Modelo de Dominio

Las entidades del dominio son **Usuario**, **Evento**, **Entrada**, **Orden** y **Pago** (Figura 5). Un usuario organizador publica muchos eventos; un evento emite muchas entradas (una por asiento o cupo); un comprador realiza muchas órdenes; cada orden contiene una o más entradas y puede tener varios intentos de pago.

Figura 5

Modelo de Dominio de EventFlow con el Dominio Core y los Subdominios



Nota. Elaboración propia en diagrams.net (`diagramas/03_modelo_dominio.drawio`). En verde, el Dominio Core.

¿Cuál es el Dominio Core y por qué no es el Login? En DDD, el Dominio Core es la parte del negocio que da la ventaja competitiva y donde conviene invertir el mejor talento y el diseño a medida; los subdominios genéricos son problemas ya resueltos que cualquier empresa tiene y que conviene comprar o delegar (Evans, 2003). Para EventFlow, el core es la **Gestión de Inventario de Entradas y Pedidos**: vender un inventario limitado y perecedero (un asiento del concierto del sábado no vale nada el domingo) a miles de personas que llegan en el mismo minuto, sin sobreventa, con reservas temporales justas y sin perder órdenes. Eso es lo que hace que un fan prefiera EventFlow frente a la competencia o la reventa. En cambio, el **Login de Usuario** es un subdominio genérico: registrarse e iniciar sesión funciona igual en cualquier aplicación y puede resolverse con un proveedor de identidad (OAuth con Google, Auth0, Amazon Cognito o Keycloak); construirlo a mano no diferencia a EventFlow y sí añade riesgo de seguridad. Los pagos son un subdominio de soporte que se delega a una pasarela especializada. La consecuencia arquitectónica es clara: los servicios de Eventos, Entradas y Pedidos se diseñan y optimizan internamente, mientras Usuarios y Pagos se integran detrás de interfaces.

Monolito vs. Microservicios

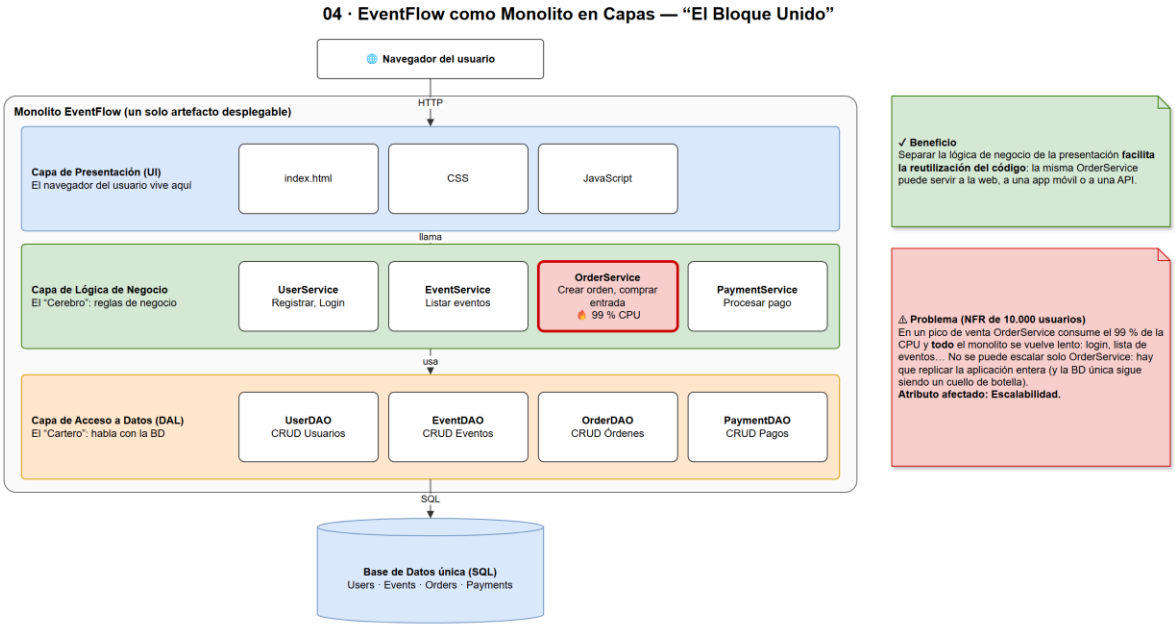
Paso 1: EventFlow como Monolito en Capas

El primer diseño es el patrón de arquitectura en capas, “el bloque unido” (Figura 6): un solo artefacto desplegable con la **capa de presentación** (el `index.html` del taller, CSS y JavaScript que corren en el navegador), la **capa de lógica de negocio** —el “cerebro”— con `UserService` (registrar, login), `EventService` (listar eventos), `OrderService` (crear orden, comprar entrada) y `PaymentService` (procesar pago), la **capa de acceso a datos (DAL)** —el

"cartero"— con `UserDAO`, `EventDAO`, `OrderDAO` y `PaymentDAO`, y una única base de datos SQL con las tablas Users, Events, Orders y Payments.

Figura 6

EventFlow como Monolito en Capas ("El Bloque Unido")



Nota. Elaboración propia en diagrams.net (`diagramas/04_monolito_capas.drawio`).

¿Por qué separar la lógica de negocio de la presentación facilita la reutilización del código? Porque las reglas ("no vender más entradas que el aforo", "una orden expira a los 10 minutos") quedan en un solo lugar que no depende de cómo se muestran. La misma `OrderService` puede ser invocada por la página web, por una futura app móvil, por una API para agencias o por un proceso nocturno, sin duplicar la regla en cada interfaz. Además, se puede rediseñar por completo la interfaz sin tocar las reglas, y probar las reglas con pruebas unitarias sin abrir un navegador. Si la regla estuviera embebida en la vista —como ocurre en el

`index.html` del taller, donde la "compra" vive en un `onclick` en línea (líneas 26 y 32)—, habría que copiarla en cada pantalla y cada copia podría divergir.

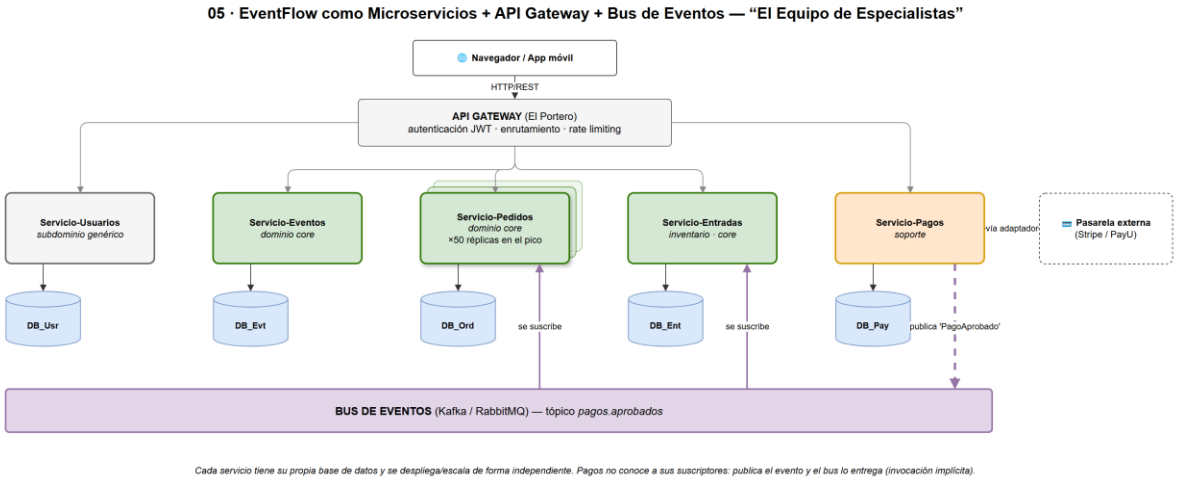
¿Qué pasa con el NFR de "10.000 usuarios"? ¿Afecta críticamente la escalabilidad? Sí. Cuando salen a la venta las entradas de un gran concierto, la carga no es uniforme: se concentra en comprar. `OrderService` puede llegar a consumir el 99 % de la CPU y, como todo comparte el mismo proceso, la misma memoria, el mismo pool de conexiones y la misma base de datos, **todo el monolito se vuelve lento**: el login, la lista de eventos y hasta las páginas estáticas. No es posible escalar solo `OrderService`; hay que replicar la aplicación entera (escalado horizontal por clonación) o comprar un servidor más grande (escalado vertical, que tiene techo y es caro). Se pagan réplicas de funciones que no lo necesitan y, aun así, la base de datos única sigue siendo el cuello de botella porque todas las réplicas compiten por las mismas filas. Tampoco hay aislamiento de fallos: un error de memoria en la compra tumba el login. Por eso el atributo críticamente afectado es la **escalabilidad**, con efecto directo en rendimiento y disponibilidad.

Paso 2: EventFlow como Microservicios

El segundo diseño, "el equipo de especialistas" (Figura 7), divide EventFlow en Servicio-Usuarios, Servicio-Eventos, Servicio-Pedidos, Servicio-Entradas (inventario) y Servicio-Pagos. Cada uno tiene su propia base de datos (DB_Usr, DB_Evt, DB_Ord, DB_Ent, DB_Pay), los clientes entran por un **API Gateway** (el portero: autenticación, enrutamiento y limitación de tasa) y los servicios se comunican de forma asíncrona por un **bus de eventos** (Kafka o RabbitMQ).

Figura 7

EventFlow como Microservicios con API Gateway, Bus de Eventos y una Base de Datos por Servicio



Nota. Elaboración propia en diagrams.net (`diagramas/05_microservicios.drawio`).

¿Cómo logramos la escalabilidad horizontal "dividiendo la aplicación en componentes más pequeños y autónomos"? Escalar horizontalmente significa agregar más instancias en lugar de agrandar una sola. Para que eso funcione, cada componente debe poder desplegarse y replicarse por separado, y sus instancias deben ser intercambiables (sin estado en memoria, con el estado en su propia base de datos o en una caché). Al dividir la aplicación por capacidades de negocio, cada servicio escala según **su** demanda: en el pico de un concierto despliego 50 copias de Pedidos y 20 de Entradas detrás de un balanceador, mientras Usuarios sigue con 2 réplicas; un autoscaler (por ejemplo, el Horizontal Pod Autoscaler de Kubernetes) añade o quita instancias según la CPU o la longitud de la cola. Como cada servicio tiene su base de datos, la carga sobre los datos también se reparte (Newman, 2021). Es la combinación de dos

ejes de escalado: descomposición funcional (dividir por servicio) y clonación (replicar solo el servicio caliente).

“Diseñar servicios con alta cohesión y bajo acoplamiento”. La **cohesión** mide qué tan relacionadas están las responsabilidades dentro de un servicio: Servicio-Pagos agrupa todo lo de pagos (autorizar, reembolsar, conciliar) y nada más, de modo que una nueva regla de pagos se implementa en un solo lugar. El **acoplamiento** mide cuánto depende un servicio de los detalles internos de otro: en bajo acoplamiento, los servicios solo se conocen por contratos estables (una API REST versionada o un evento `PagoAprobado` con un esquema definido) y **nunca comparten base de datos**. El beneficio para la mantenibilidad es concreto: el equipo de Pagos puede cambiar su base de datos, refactorizar o desplegar un martes a las 3 p. m. sin avisarle al equipo de Usuarios, porque el contrato no cambió. Alta cohesión y bajo acoplamiento reducen el “radio de impacto” de cada cambio, que es exactamente lo que SAAM mide.

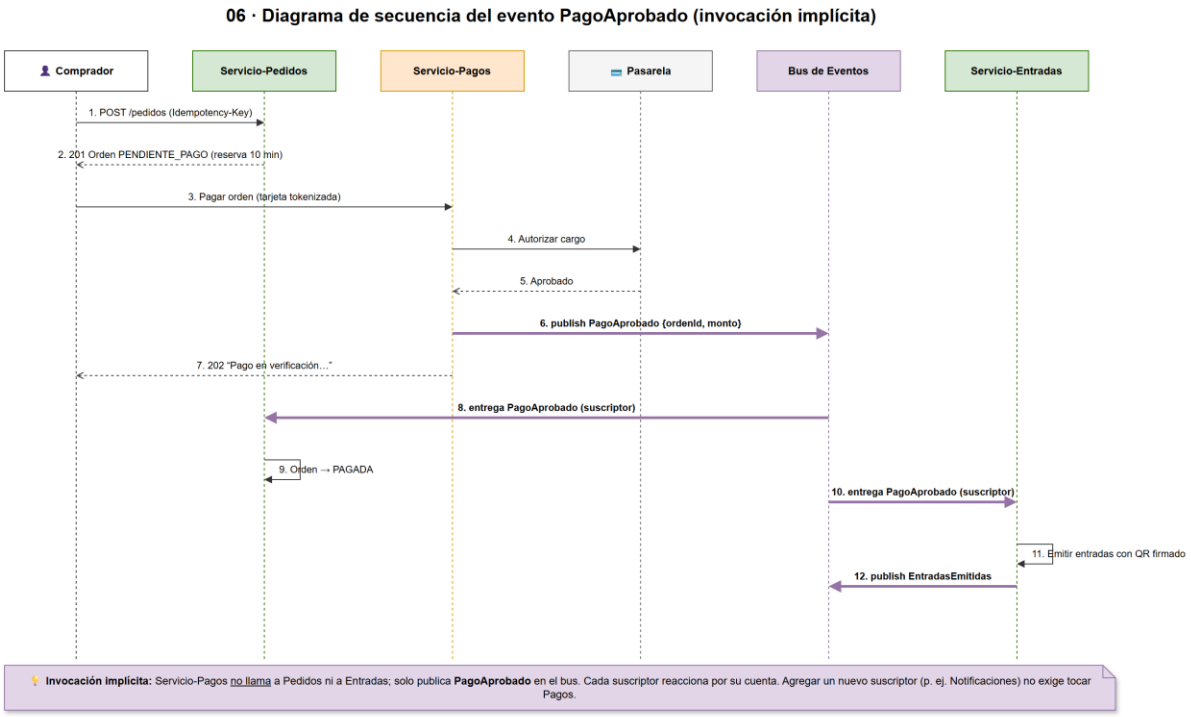
Refutar el mito: ¿son fáciles de diseñar y desarrollar como una aplicación monolítica? No. La afirmación es incorrecta porque los microservicios cambian la complejidad de lugar: del código a la red y a la operación. Una llamada que en el monolito era una invocación de método en memoria ahora cruza la red y puede fallar, tardar o duplicarse; los datos están distribuidos, así que una compra que antes era una transacción ACID ahora requiere consistencia eventual, patrones como saga y outbox, e idempotencia; depurar un error exige trazas distribuidas, logs centralizados y métricas por servicio; y cada servicio necesita su pipeline, su contenedor, su orquestación y versionado de contratos. Además, si los límites entre servicios se eligen mal, el resultado es un “monolito distribuido”: todo el acoplamiento del monolito más la latencia de la red. Por eso Newman (2021) recomienda adoptar microservicios solo cuando hay un motivo claro —como el NFR de 10.000 usuarios— y no como punto de partida por moda.

Paso 3: Arquitectura de Eventos

Cuando un pago es aprobado, ¿cómo se entera el servicio de Entradas? El Servicio-Pagos **publica** un evento `PagoAprobado` en el bus, y los servicios de Pedidos y Entradas están **suscritos** a ese tipo de evento y reaccionan (Figura 8). Esto es la "invocación implícita de una herramienta cuando se produce un evento": en lugar de que Pagos llame explícitamente a `Pedidos.confirmar()` y a `Entradas.emitir()`, se limita a anunciar que algo ocurrió; el bus invoca a todos los componentes que registraron interés. El publicador no sabe quién lo escucha ni cuántos son (Garlan & Shaw, 1994). Así, Pagos no se bloquea esperando a los demás y, si Entradas está caído, el evento espera en el bus hasta que vuelva.

Figura 8

Diagrama de Secuencia del Evento PagoAprobado



Nota. Elaboración propia en diagrams.net (`diagramas/06_secuencia_pago_aprobado.drawio`); versión Mermaid en `.mmd`.

Ejemplo concreto. El comprador paga la orden 123. Pagos publica `PagoAprobado {ordenId: 123, monto: 360000}`. Pedidos lo recibe y marca la orden como PAGADA; Entradas lo recibe y emite dos QR firmados; un mes después el negocio pide enviar un correo de confirmación y se crea un Servicio-Notificaciones que simplemente se suscribe al mismo evento: **no se modifica ni una línea de Pagos**. El propio taller contiene otro ejemplo de invocación implícita: el archivo `ci.yml` declara `on: push` (línea 5), de modo que el evento "push a main" invoca automáticamente la herramienta GitHub Actions sin que nadie la llame explícitamente.

Figura 9

Diagramas del Taller Abiertos en diagrams.net

PENDIENTE: fig09_drawio_evidencia.png

Editor de diagrams.net (o Miro) mostrando uno de los .drawio, con la paleta y las páginas visibles

Nota. Captura de pantalla de diagrams.net.

DevOps, Infraestructura como Código y Deuda Técnica

DevOps es una cultura y un conjunto de prácticas que unen desarrollo y operaciones en un ciclo continuo —planificar, codificar, construir, probar, liberar, desplegar, operar y monitorear— apoyado en la automatización, la retroalimentación rápida y el aprendizaje continuo (Kim et al., 2016). Para simular ese ciclo trabajé exactamente con los tres archivos del enunciado.

Paso 1: El Código en GitHub

Creé el repositorio `taller-arquitectura` con `index.html` (el monolito "malo"), `.github/workflows/ci.yml` (el pipeline) y `main.tf` (la infraestructura), copiados sin modificar, más un `openapi.yaml` opcional para Swagger Hub. El primer commit sube los archivos y el segundo cambia el año del copyright de 2025 a 2026 en la línea 36 de `index.html` para disparar el pipeline (Figura 10).

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>EventFlow - Venta de Entradas</title>
    <style>
        body { font-family: Arial, sans-serif; padding: 20px; background-color: #f4f4f4;
}
        .container { max-width: 600px; margin: auto; background: #fff; padding: 20px;
border-radius: 8px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }
        h1 { color: #333; }
        button { background-color: #007bff; color: white; padding: 10px 15px; border:
none; border-radius: 4px; cursor: pointer; }
        button:hover { background-color: #0056b3; }

        <style>
            .evento { border-bottom: 1px solid #ddd; padding-bottom: 10px; margin-bottom:
10px; }
        </style>
    </style>
</head>
<body>
    <div class="container">
        <h1>Bienvenido a EventFlow</h1>
        <p>Tu plataforma para comprar entradas a los mejores eventos.</p>

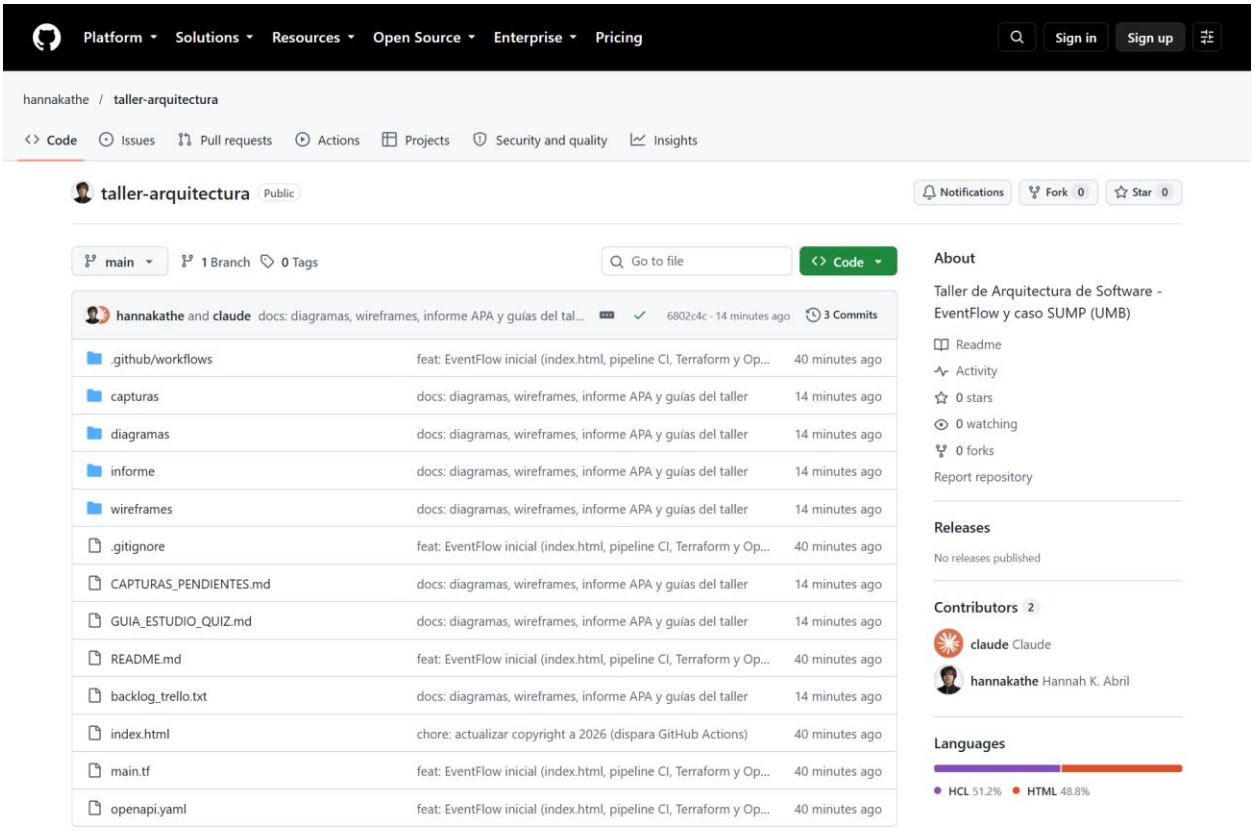
        <div class="evento">
            <h2>Concierto de Rock</h2>
            <p>Fecha: 30 de Diciembre</p>
            <button onclick="alert('¡Entrada comprada! (No realmente)')">Comprar
Entrada</button>
        </div>

        <div class="evento">
            <h2>Feria de Tecnología</h2>
            <p>Fecha: 15 de Enero</p>
            <button onclick="alert('¡Entrada comprada! (No realmente)')">Comprar
Entrada</button>
        </div>

        <footer>
            <p>&copy; 2026 EventFlow. Todos los derechos reservados.</p>
        </footer>
    </div>
</body>
</html>
```

Figura 10

Repositorio taller-arquitectura en GitHub



Nota. Captura de pantalla de GitHub.

Paso 2: Análisis de Deuda Técnica con SonarCloud

Inicié sesión en SonarCloud con la cuenta de GitHub, importé el repositorio y ejecuté el análisis automático (Figuras 11 y 12). La deuda técnica es la metáfora de tomar atajos que funcionan hoy pero que generan "intereses": cada cambio futuro cuesta más hasta que se paga la deuda refactorizando. Los **malos olores** (*code smells*) son los síntomas visibles de esa deuda en el código: no rompen la funcionalidad, pero dificultan entenderlo y cambiarlo (Fowler, 2018). SonarCloud los clasifica en la dimensión de mantenibilidad, estima el tiempo de remediación y asigna una calificación de la A a la E (SonarSource, s.f.).

Al revisar el `index.html` del taller, estos son los malos olores intencionales que el análisis pone en evidencia (la lista exacta de reglas depende del perfil de calidad activo, por defecto "Sonar way"):

- **`<style>` anidado dentro de otro `<style>` (líneas 6 a 16).** Es HTML/CSS inválido: el navegador interpreta el texto `<style>` interno como CSS basura, de modo que la regla `.evento` puede no aplicarse. Es un error latente y confuso para quien mantenga el archivo.
- **CSS mezclado con el HTML.** Los estilos embebidos impiden reutilizarlos en otras páginas y cachearlos; lo mantenible es una hoja `styles.css` externa.
- **JavaScript en línea en atributos `onclick` (líneas 26 y 32).** Mezcla comportamiento con estructura, es incompatible con una política de seguridad de contenido (CSP) estricta y no se puede probar de forma aislada.
- **Código duplicado.** El mismo literal `alert('¡Entrada comprada! (No realmente)')` aparece dos veces; un cambio en el mensaje exige editar ambos lugares.
- **Uso de `alert()`.** Bloquea la interfaz y es una mala práctica de experiencia de usuario; además, la "compra" no hace nada real (lógica de negocio simulada en la vista).
- **Datos "quemados" en el HTML** (eventos, fechas y año del copyright): cada nuevo evento obliga a editar y desplegar el código.

¿Por qué esto es "deuda de código"? Porque ninguna de estas decisiones impide que la página se vea hoy, pero todas encarecen el mañana: agregar un tercer evento implica copiar otro bloque con otro `onclick`; cambiar el estilo de los botones exige tocar el HTML; introducir una CSP rompe los botones. El "capital" de la deuda es el atajo y el "interés" es el tiempo extra de cada cambio, que SonarCloud cuantifica en minutos de remediación.

Figura 11

Dashboard de SonarCloud del Repositorio taller-arquitectura

PENDIENTE: fig11_sonarcloud_dashboard.png

Overview del proyecto: Quality Gate, Maintainability (Code Smells, Technical Debt), Reliability y Security

Nota. Captura de pantalla de SonarCloud.

Figura 12

Lista de Code Smells Detectados en index.html

PENDIENTE: fig12_sonarcloud_code_smells.png

Pestaña Issues filtrada por Type = Code Smell, con los hallazgos de index.html

Nota. Captura de pantalla de SonarCloud.

Paso 3: El Ciclo DevOps con GitHub Actions

El pipeline del taller (`ci.yml`) se reproduce a continuación. Al hacer el commit del cambio de copyright y el push a `main`, GitHub detectó el archivo en `.github/workflows/` y ejecutó el flujo automáticamente (Figuras 13 a 15).

```
# Archivo: .github/workflows/ci.yml

name: Taller CI Pipeline # Nombre del pipeline

on:
  push: # Se activa con CADA push (commit)
    branches: [ "main" ] # Solo en la rama principal
  workflow_dispatch: # Permite iniciarlo manualmente

jobs:
  # 1. El trabajo de "Construcción" (Build)
```

```
  build:
    runs-on: ubuntu-latest # Se ejecuta en una máquina virtual de Linux

    steps:
      - name: 1. Checkout del código
        uses: actions/checkout@v4 # Acción oficial para descargar el código del repo

      - name: 2. Simular una construcción (Build)
        run: echo "Construyendo el proyecto... (aquí iría 'npm run build', etc.)"

      - name: 3. Simular guardado de artefacto
        run: |
          mkdir -p build
          cp index.html build/
          echo "Archivos 'construidos' movidos a la carpeta /build"

  # 2. El trabajo de "Pruebas" (Test)
  test:
    needs: build # Este trabajo NO empieza hasta que 'build' termine
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout del código
        uses: actions/checkout@v4

      - name: 2. Simular pruebas (Linting)
        run: |
          echo "Ejecutando pruebas..."
          # Simulación de una prueba que busca la palabra "Rock"
          grep -q "Rock" index.html
          echo "Prueba de 'Linting' pasada."

      - name: 3. Simular pruebas de unidad (Unit Test)
        run: |
          echo "Ejecutando pruebas de unidad..."
          # Simulación de una prueba que busca el copyright
          grep -q "EventFlow" index.html
          echo "Prueba de 'Copyright' pasada."

      - name: 4. ¡Despliegue simulado!
        run: echo "¡Pruebas exitosas! Listo para desplegar."
```

Explicación del ciclo paso a paso (GitHub, s.f.):

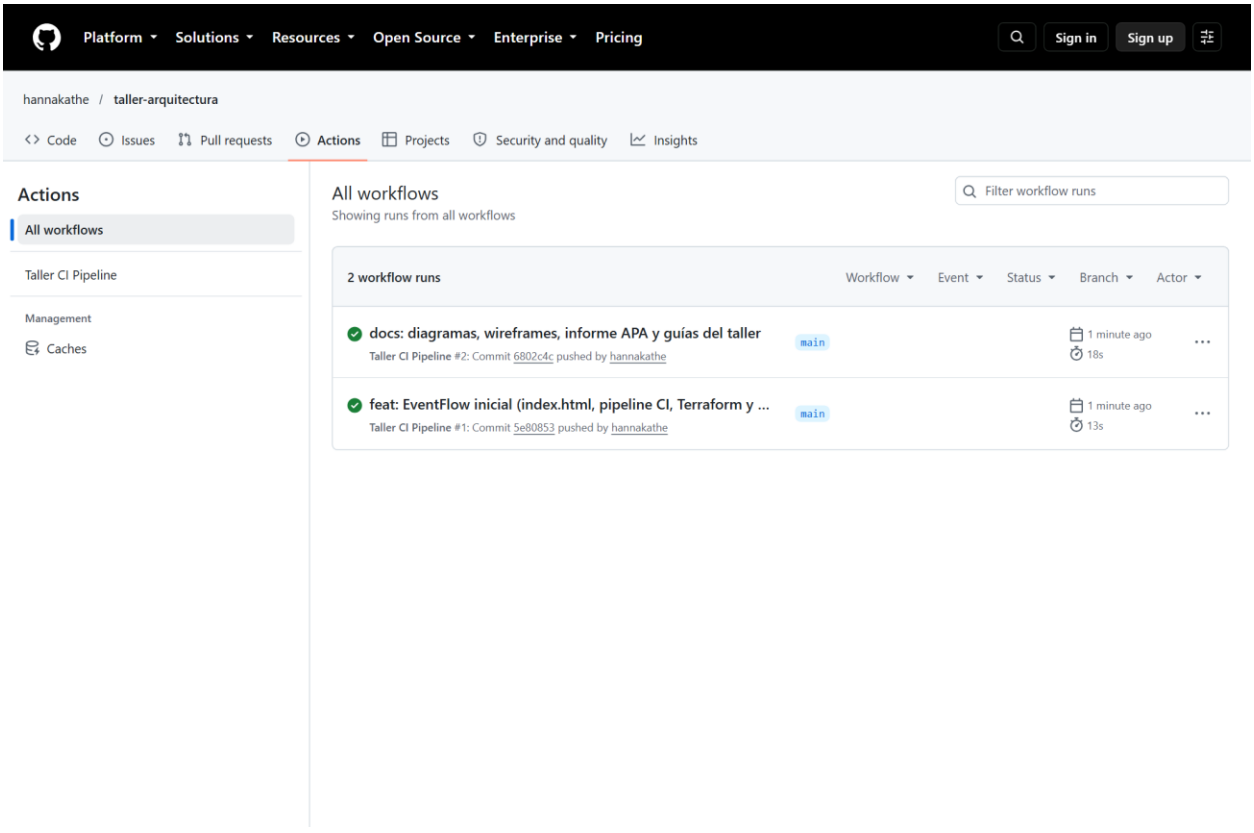
- **Codifican → commit/push.** El desarrollador cambia `2025` por `2026` y hace push a `main`. El disparador `on: push: branches: ["main"]` (líneas 5 a 7) convierte ese evento en una ejecución del pipeline; `workflow_dispatch` permite lanzarlo también a mano.
- **Se aprovisiona el entorno.** Para el job `build` GitHub crea una máquina virtual limpia `ubuntu-latest` (línea 13). Que sea limpia es clave: el resultado no depende del computador del desarrollador.

- **Se extrae el código.** El paso `actions/checkout@v4` descarga el commit exacto que disparó el flujo.
- **Se construye.** Los pasos 2 y 3 simulan el build y empaquetan `index.html` en `build/` (el "artefacto").
- **Se ejecutan pruebas.** El job `test` declara `needs: build` (línea 30): no empieza si el build falla. En otra VM nueva vuelve a hacer checkout y ejecuta dos verificaciones: `grep -q "Rock" index.html` (simulación de linting) y `grep -q "EventFlow" index.html` (simulación de prueba unitaria). `grep -q` devuelve código de salida 1 si no encuentra el texto, y cualquier código distinto de 0 marca el paso en rojo.
- **Falla o pasa.** Si todo pasa se ejecuta el "despliegue simulado"; si algo falla, el despliegue no ocurre y GitHub notifica el fallo con el log exacto. Por ejemplo, si alguien borra "Rock" del HTML, el pipeline queda en rojo antes de llegar a producción.

Esto materializa el principio de "automatización para crear procesos repetitivos y fiables": el mismo commit produce siempre el mismo resultado, sin pasos manuales que alguien pueda olvidar, con retroalimentación en minutos y con trazabilidad (quién cambió qué y si pasó las pruebas). Como observación de ingeniería, el artefacto `build/` del primer job no llega al segundo, porque cada job corre en una VM distinta; en un pipeline real se compartiría con `actions/upload-artifact` y `actions/download-artifact`. Además, la "prueba de copyright" solo busca la palabra EventFlow y no el año, por lo que no detectaría un copyright desactualizado.

Figura 13

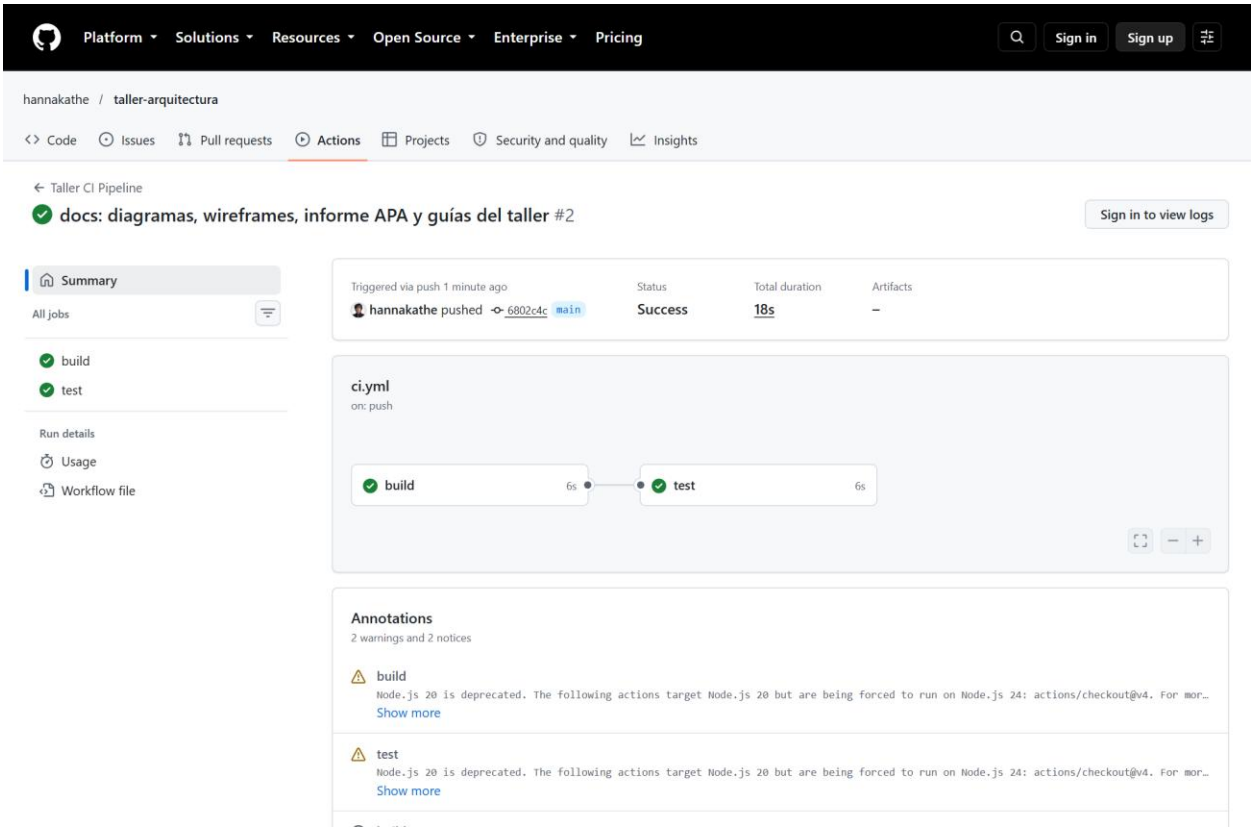
Pestaña Actions con las Ejecuciones del Taller CI Pipeline



Nota. Captura de pantalla de GitHub Actions.

Figura 14

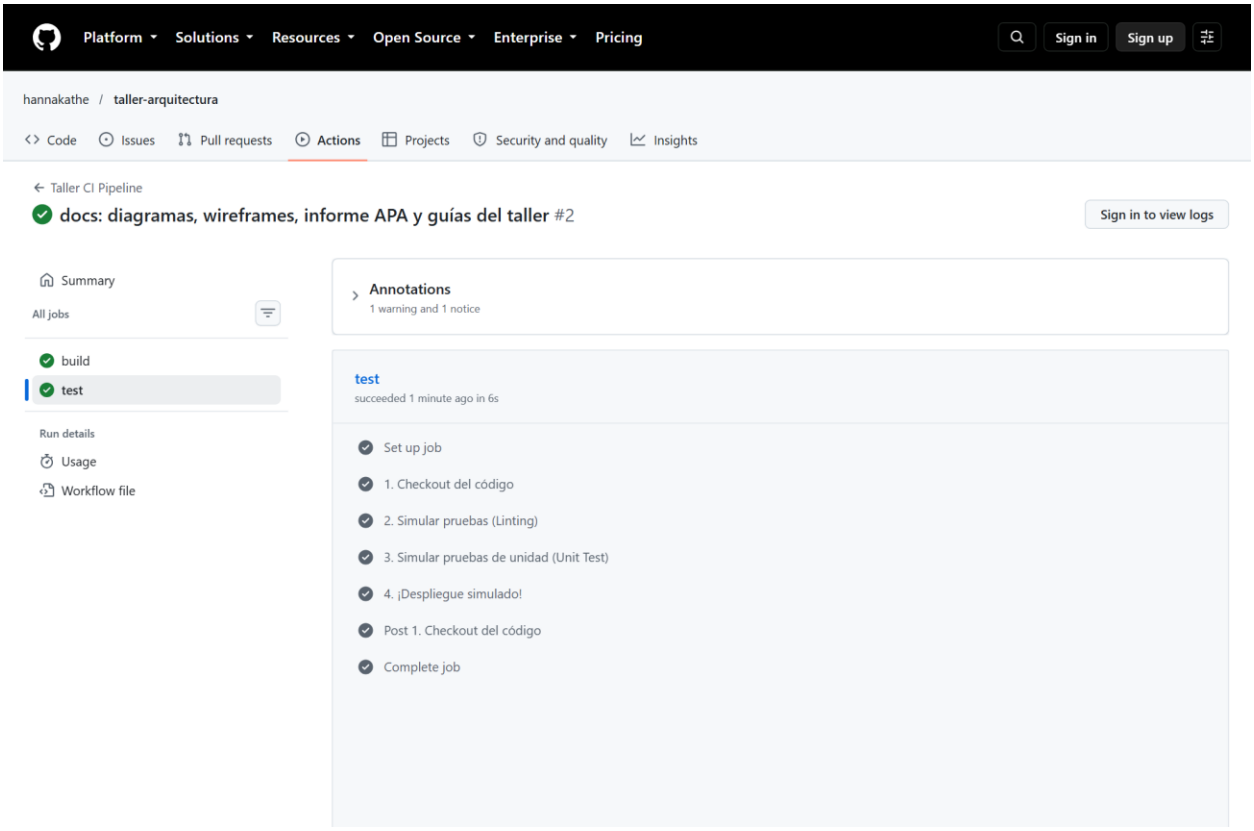
Detalle de la Ejecución con los Jobs build y test



Nota. Captura de pantalla de GitHub Actions.

Figura 15

Pasos del Job test (Checkout, Linting, Unit Test y Despliegue Simulado) Ejecutados con Éxito



Nota. Captura de pantalla de GitHub Actions.

Paso 4: Infraestructura como Código con Terraform en Google Cloud Shell

La infraestructura como código (IaC) consiste en definir servidores, redes y reglas en archivos versionados en lugar de crearlos a mano en una consola. El `main.tf` del taller declara tres recursos de Google Cloud: una red VPC `taller-arquitectura-vpc` (línea 25), una regla de firewall que permite HTTP por el puerto 80 desde cualquier IP (líneas 31 a 40) y una máquina virtual `e2-micro` llamada `eventflow-web-server` con Debian 11 e IP pública (líneas 43 a 60). En Cloud Shell cloné el repositorio, reemplacé `tu-proyecto-de-gcloud` (línea 18) por mi Project ID y ejecuté los comandos sin aplicar (Figuras 16 y 17).

```
# Archivo: main.tf
#
# Configuración de Terraform para el Taller de Arquitectura
# ¡OJO! Esto es solo para demostración en Cloud Shell.
# No creará recursos reales a menos que se aplique (no lo haremos).

terraform {
  required_providers {
    # Usaremos el proveedor de Google Cloud
    google = {
      source  = "hashicorp/google"
      version = "5.10.0"
    }
  }
}

provider "google" {
  project = "tu-proyecto-de-gcloud" # Esto es solo un placeholder
  region  = "us-central1"
  zone    = "us-central1-c"
}

# Definición de "infraestructura como código" [cite: 37, 100]
# 1. Una red virtual para nuestra app
resource "google_compute_network" "vpc_network" {
  name = "taller-arquitectura-vpc"
  auto_create_subnetworks = true
}

# 2. Una regla de firewall para permitir tráfico web (HTTP)
resource "google_compute_firewall" "default" {
  name    = "taller-firewall-allow-http"
  network = google_compute_network.vpc_network.name

  allow {
    protocol = "tcp"
    ports    = ["80"] # Puerto HTTP
  }
  source_ranges = ["0.0.0.0/0"] # Permitir desde cualquier IP
}

# 3. Un servidor web (máquina virtual)
resource "google_compute_instance" "web_server" {
  name         = "eventflow-web-server"
  machine_type = "e2-micro" # Una máquina pequeña y barata
  zone         = "us-central1-c"

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }

  network_interface {
    network = google_compute_network.vpc_network.name
    access_config {
      # Asigna una IP pública (efímera)
    }
  }
}
```

- **`terraform init`** prepara el directorio de trabajo: lee el bloque `required_providers`, descarga el proveedor `hashicorp/google` en la versión fijada 5.10.0 (líneas 10 a 13) dentro de `.terraform/`, crea el archivo de bloqueo `.terraform.lock.hcl` para que todos usen la misma versión e inicializa el backend de estado (local por defecto). No toca la nube (HashiCorp, s.f.).
- **`terraform plan`** compara la configuración deseada con el estado real (vacío, porque nunca se aplicó) y muestra el plan de ejecución: `Plan: 3 to add, 0 to change, 0 to destroy`, con el detalle de la red, el firewall y la VM. Tampoco crea nada; es una revisión previa, como un *diff* de infraestructura.
- **`terraform apply`** —que no se ejecutó, por indicación del taller— sería el único comando que crea los recursos y genera costos.

Esto es “una función principal de DevOps que permite gestionar la escala y la velocidad a la que deben aprovisionarse los entornos”: el mismo archivo crea en minutos un entorno idéntico de pruebas, de staging o diez réplicas para un pico de ventas, se revisa en un pull request como cualquier código y elimina la “configuración artesanal” que nadie sabe reproducir. Como auditora, anoto dos deudas de infraestructura en el propio `main.tf`: `source_ranges = ["0.0.0.0/0"]` es aceptable para HTTP público, pero debería restringirse cuando se agreguen otros puertos, y la imagen `debian-11` ya está fuera de su soporte regular, por lo que convendría migrar a `debian-12`.

Figura 16

Ejecución de terraform init en Google Cloud Shell

PENDIENTE: fig16_cloudshell_terraform_init.png

Terminal de Cloud Shell con “Terraform has been successfully initialized!”

Nota. Captura de pantalla de Google Cloud Shell.

Figura 17

Ejecución de terraform plan en Google Cloud Shell

PENDIENTE: fig17_cloudshell_terraform_plan.png

Salida de terraform plan con los 3 recursos y “Plan: 3 to add, 0 to change, 0 to destroy”

Nota. Captura de pantalla de Google Cloud Shell.

Diseño de la API en Swagger Hub

Como complemento, describí en `openapi.yaml` (OpenAPI 3.0) una API mínima de los servicios de Eventos y Pedidos: `GET /eventos` (la HU-01), `GET /eventos/{id}/disponibilidad` con objetivo p95 ≤ 200 ms y `POST /pedidos` con la cabecera obligatoria `Idempotency-Key`, que evita órdenes duplicadas por reintentos (Figura 18). Diseñar el contrato primero (*API-first*) es lo que permite el bajo acoplamiento entre servicios discutido antes.

Figura 18

Especificación OpenAPI de EventFlow en Swagger Hub

PENDIENTE: fig18_swagger_hub.png

Swagger Hub con openapi.yaml importado: endpoints de Eventos y Pedidos visibles

Nota. Captura de pantalla de Swagger Hub (opcional).

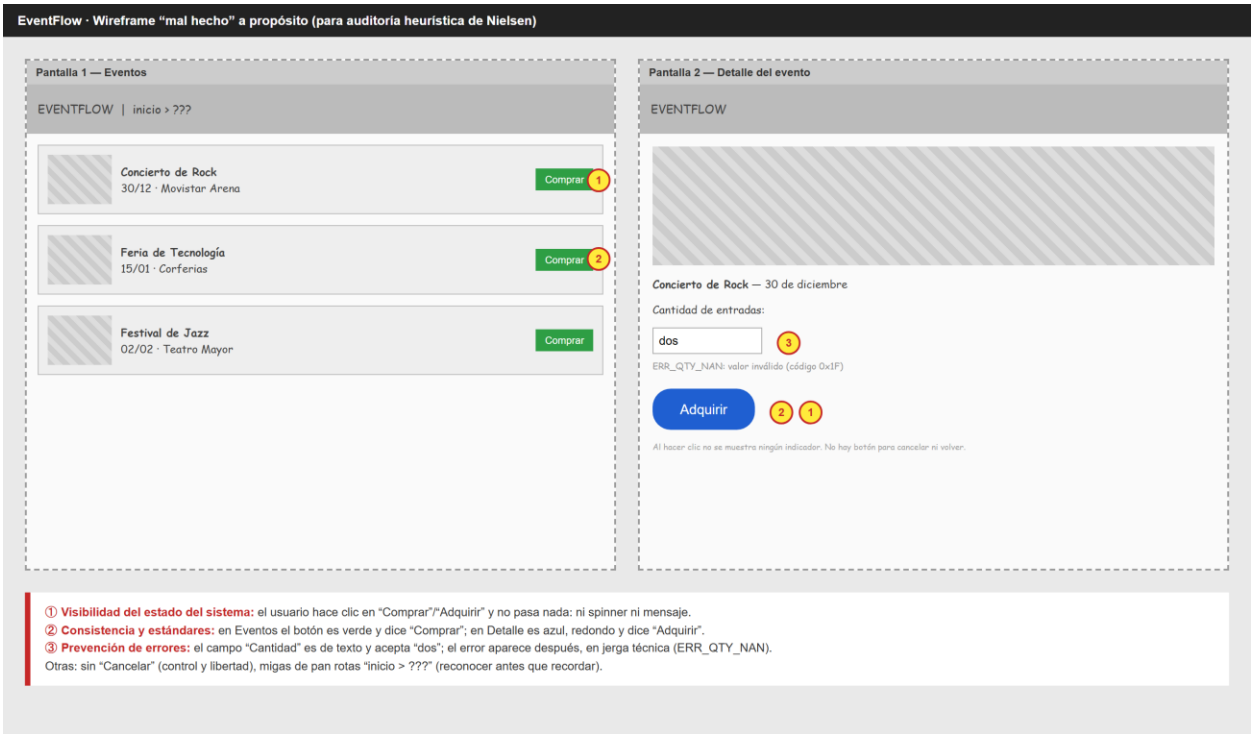
Evaluación de la Arquitectura y de la Experiencia de Usuario

Auditoría UX con las Heurísticas de Nielsen

Diseñé en Figma un wireframe de EventFlow intencionalmente mal hecho, en escala de grises, con dos pantallas: *Eventos* y *Detalle* (Figuras 19 y 20). Los números en amarillo marcan las violaciones.

Figura 19

Wireframe de EventFlow Intencionalmente Mal Diseñado



Nota. Elaboración propia (`wireframes/eventflow_malo.html`, exportado a PNG; versión móvil en `eventflow_malo_movil.png`).

Figura 20

Wireframe Mal Diseñado Recreado en Figma

PENDIENTE: fig20_figma_wireframe_malo.png

Frame "EventFlow — Malo" en Figma con las pantallas Eventos y Detalle y los marcadores 1–3

Nota. Captura de pantalla de Figma.

① “El usuario hace clic en ‘Comprar’ y no pasa nada”: viola la Visibilidad del estado del sistema. Esta heurística exige que el sistema mantenga informado al usuario de lo que ocurre, con retroalimentación oportuna. Sin spinner ni mensaje, el usuario no sabe si el clic se registró, si la reserva está en curso o si falló, y reacciona haciendo clic otra vez (riesgo de compras duplicadas) o abandonando. La corrección es mostrar inmediatamente “Reservando tu entrada…” con un indicador de progreso, deshabilitar el botón y confirmar el resultado.

② Botón verde “Comprar” en Eventos y azul “Adquirir” en Detalle: viola Consistencia y estándares. El usuario no debería preguntarse si palabras, colores o acciones distintas significan lo mismo. Dos etiquetas y dos estilos para la misma acción obligan a reinterpretar la interfaz y hacen dudar de si “Adquirir” es otra cosa (¿reservar?, ¿suscribirse?). La corrección es un único componente de botón primario, con la misma etiqueta (“Comprar entrada”), color y forma en todas las pantallas.

③ El campo “Cantidad de entradas” acepta “dos” y luego da error: viola la Prevención de errores (y, por el mensaje técnico `ERR_QTY_NAN`, también la heurística de ayudar a reconocer, diagnosticar y recuperarse de errores). Es mejor un diseño que impida el error que un buen mensaje después. La corrección es un input numérico (`type="number"`, `min="1"`, `max="6"`) con botones − y +, que haga imposible escribir texto.

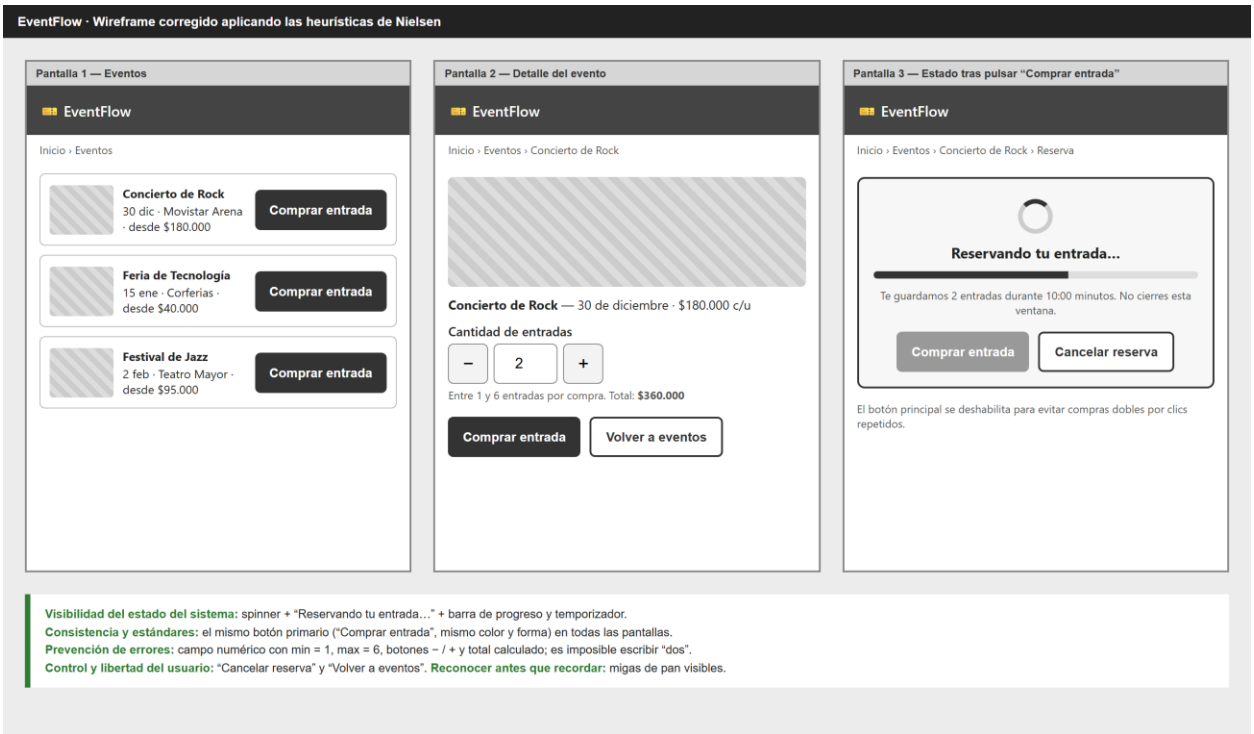
¿“Agradar visualmente al usuario” o “facilitar la navegación del usuario” son heurísticas? No, no forman parte de las diez originales de Nielsen (1994b). La razón es de fondo: una heurística es un principio **evaluable** que permite señalar un problema concreto en una pantalla y decir qué regla incumple. “Agradar visualmente” es un juicio estético subjetivo —lo que agrada a una persona no agrada a otra— y Nielsen no pide belleza, sino que los diálogos no contengan información irrelevante (*diseño estético y minimalista*, que trata de relevancia y señal

frente a ruido, no de gusto). “Facilitar la navegación” es un **objetivo o resultado** que se logra cumpliendo varias heurísticas a la vez —*reconocer antes que recordar* (opciones y ubicación visibles, como las migas de pan), *consistencia y estándares* y *control y libertad del usuario*—, no una regla en sí misma. Las dos heurísticas reales más cercanas son, por tanto, “diseño estético y minimalista” y “reconocer antes que recordar”.

A partir de la auditoría construí el wireframe corregido (Figuras 21 y 22): botones consistentes, input numérico con límites y total calculado, una tercera pantalla con el estado “Reservando tu entrada…” (spinner, barra de progreso, temporizador de 10 minutos y botón deshabilitado) y la salida de emergencia “Cancelar reserva”.

Figura 21

Wireframe de EventFlow Corregido con las Heurísticas de Nielsen



Nota. Elaboración propia (`wireframes/eventflow_corregido.html`, exportado a PNG; versión móvil en `eventflow_corregido_movil.png`).

Figura 22

Wireframe Corregido Recreado en Figma

PENDIENTE: fig22_figma_wireframe_corregido.png

Frame “EventFlow — Corregido” en Figma con las 3 pantallas (Eventos, Detalle, Reservando…)

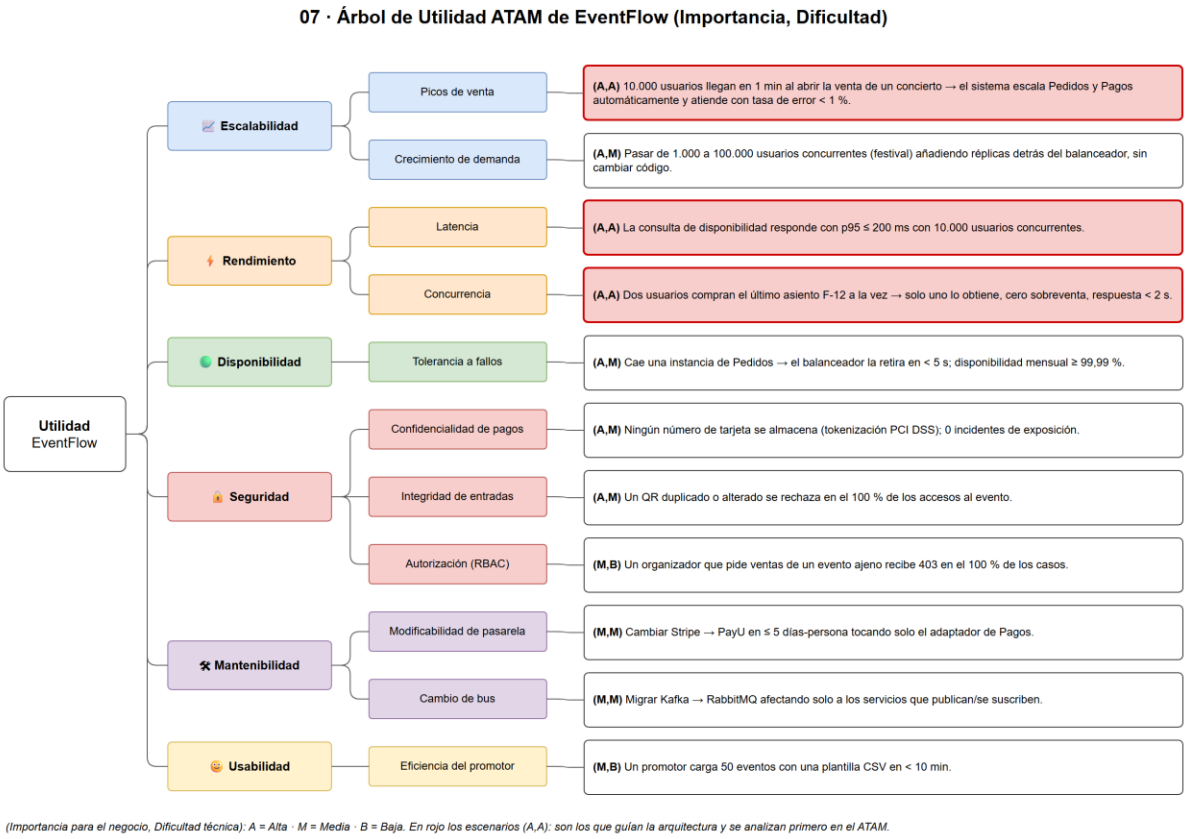
Nota. Captura de pantalla de Figma.

Simulación 1: El ATAM

Escenario. “Soy el cliente y estamos discutiendo la capacidad del sistema para manejar picos de tráfico durante eventos de ventas especiales”. **¿Qué atributo de calidad estamos analizando? La escalabilidad**, porque la pregunta no es qué tan rápido responde el sistema con poca carga (rendimiento), sino si puede mantener esa respuesta cuando la carga se multiplica, agregando recursos de manera proporcional. Este escenario es un *driver* arquitectónico: un requisito tan importante y difícil que por sí solo condiciona la elección del estilo. Lo ubiqué en el árbol de utilidad con prioridad (A,A) junto con los escenarios de latencia y de concurrencia (Figura 23).

Figura 23

Árbol de Utilidad ATAM de EventFlow



Nota. Elaboración propia en diagrams.net (`diagramas/07_arbol_utilidad_atam.drawio`). (Importancia, Dificultad): A = alta, M = media, B = baja.

Análisis del monolito: responde mal. Si la lógica de "Comprar" en la capa de negocio consume el 99 % de la CPU, todo el sistema se degrada o se cae —login, búsqueda, catálogo— porque comparten proceso, memoria y base de datos. La única táctica disponible es replicar el monolito completo, que es costosa y no elimina el cuello de botella de la base de datos única. En

términos de ATAM, “base de datos y proceso compartidos” es un **punto de sensibilidad** para la escalabilidad y un **riesgo**: el sistema puede no cumplir el NFR de 10.000 usuarios por minuto.

Análisis de los microservicios: responde bien, con condiciones. Se escalan horizontalmente solo Pedidos y Pagos (y Entradas), mientras Usuarios y Eventos siguen funcionando; un fallo en Pedidos no tumba el catálogo. Pero un ATAM honesto también registra los nuevos riesgos: el API Gateway y el bus de eventos se vuelven componentes críticos que deben replicarse, la base de datos de inventario concentra la contención por los asientos más populares, y la consistencia pasa a ser eventual. El resultado es un **punto de trade-off**: ganamos escalabilidad y disponibilidad a cambio de complejidad operativa y consistencia inmediata.

Tabla 2

Hallazgos de la simulación ATAM para el escenario de pico de tráfico

Decisión arquitectónica	Tipo de hallazgo	Atributo(s)	Comentario
Monolito con BD única	Riesgo	Escalabilidad, disponibilidad	Un pico en OrderService degrada todo el sistema
Servicios sin estado detrás de balanceador	Punto de sensibilidad (+)	Escalabilidad	Permite escalar solo Pedidos/Pagos
Bus de eventos asíncrono	Punto de trade-off	Disponibilidad (+) / Consistencia (−)	Desacopla, pero introduce consistencia eventual
API Gateway único	Riesgo (mitigable)	Disponibilidad	Debe replicarse en varias zonas
Una BD por servicio	No-riesgo	Mantenibilidad, escalabilidad	Decisión validada: aísla datos y carga

Nota. Elaboración propia.

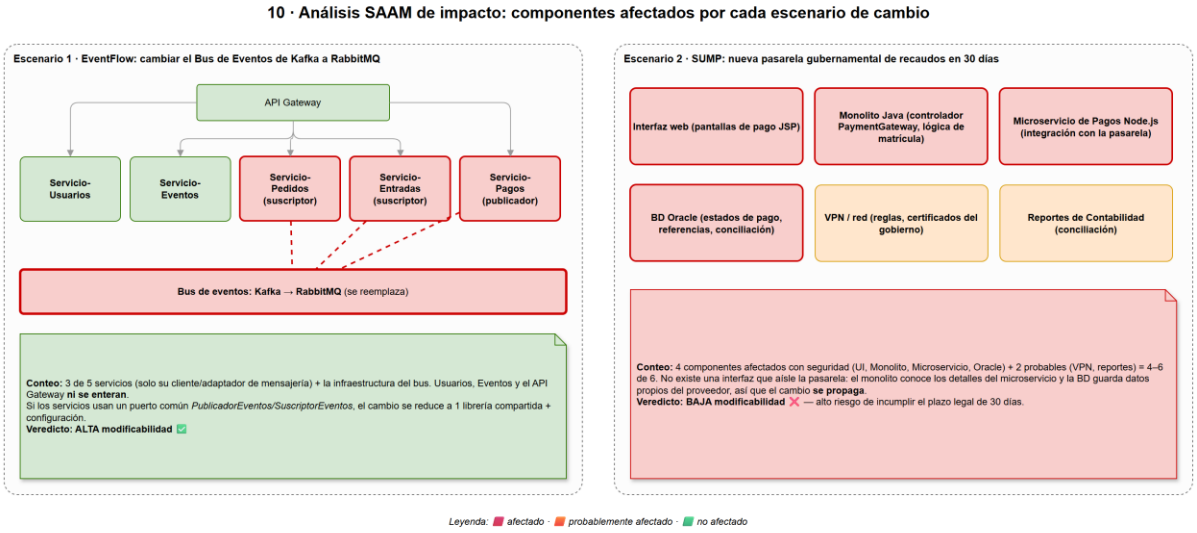
Con esto queda hecho un ATAM en miniatura: usé un escenario de negocio para identificar un atributo de calidad (escalabilidad) y analicé cómo responden a él dos estilos arquitectónicos, documentando riesgos, puntos de sensibilidad y trade-offs (Kazman et al., 2000).

Simulación 2: El SAAM

Escenario. “Soy el Arquitecto Jefe y quiero evaluar cómo un cambio en el protocolo de comunicación afecta al sistema: ¿qué tan sensible es nuestra arquitectura de microservicios si cambiamos el bus de eventos de Kafka a RabbitMQ?”. **¿Qué estamos haciendo? Un análisis de sensibilidad** dentro de un SAAM: tomamos un escenario de cambio indirecto (la arquitectura no lo soporta sin modificaciones) y contamos qué componentes se tocan (Figura 24, panel izquierdo).

Figura 24

Análisis SAAM del Impacto de los Escenarios de Cambio en EventFlow y en el SUMP



Nota. Elaboración propia en diagrams.net (`diagramas/10_saam_impacto.drawio`). Rojo: afectado; naranja: probablemente afectado; verde: no afectado.

Análisis. Solo cambian los servicios acoplados al bus: Servicio-Pagos, que publica `PagoAprobado`, y Servicio-Pedidos, que lo escucha (en mi diseño también Servicio-Entradas, que es otro suscriptor). En cada uno se modifica únicamente el cliente de mensajería (la librería

de Kafka por la de RabbitMQ y la configuración de colas e intercambios), no su lógica de negocio. Servicio-Usuarios, Servicio-Eventos y el API Gateway “ni se enteran”. Si además los servicios publican y consumen a través de una interfaz propia (`PublicadorEventos`/`SuscriptorEventos`), el cambio se reduce a una librería compartida.

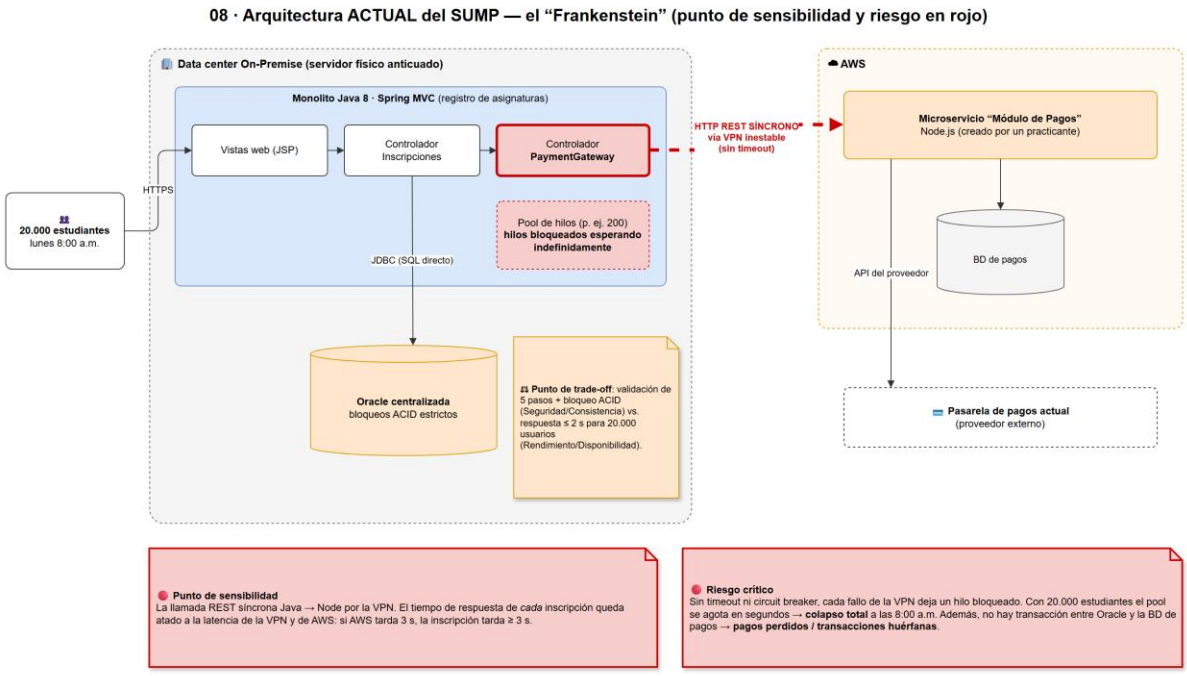
Conclusión: el impacto queda aislado en los componentes que usan el bus, por lo que la arquitectura de microservicios tiene **alta modificabilidad** frente a este escenario, y el cliente del bus es su punto de sensibilidad. Eso es un SAAM en acción: identificar los puntos de sensibilidad de la arquitectura frente a un cambio específico.

Ejercicio Práctico 2: Auditoría Integral del Caso SUMP

Como nueva firma consultora de la Universidad Nacional de los Andes (ficticia), audité el SUMP, el sistema con el que los estudiantes inscriben asignaturas y pagan la matrícula. Los síntomas reportados son tres: colapso cada semestre en la semana de inscripciones, pagos perdidos según Contabilidad y 5.000 llamadas diarias al centro de ayuda. Cada síntoma corresponde a un atributo de calidad: disponibilidad/rendimiento, consistencia/fiabilidad y usabilidad.

Figura 25

Arquitectura Actual del SUMP con su Punto de Sensibilidad y su Riesgo Crítico



Nota. Elaboración propia en diagrams.net (`diagramas/08_sump_actual.drawio`), a partir de la descripción del caso.

Módulo A: Aplicación de ATAM

El Conflicto (Trade-off) Principal

El Vicerrector Académico exige **disponibilidad y rendimiento**: 20.000 estudiantes inscribiendo simultáneamente a las 8:00 a. m. con tiempos de carga de máximo 2 segundos. El Director Financiero exige **seguridad y consistencia**: validación de 5 pasos, confirmación criptográfica y bloqueo estricto ACID en la base de datos, "aunque sea lento". Con la arquitectura actual ambos requisitos chocan porque **la verificación del pago está dentro del camino crítico de la inscripción**: el monolito Java, para saber si el estudiante pagó, hace una llamada HTTP síncrona al microservicio de AWS por una VPN inestable, y mientras tanto mantiene ocupado un hilo y, potencialmente, bloqueos en Oracle. Cada paso de seguridad adicional (5 validaciones, firma criptográfica, bloqueos más amplios y largos) suma latencia a **cada** inscripción y alarga el tiempo que los recursos permanecen retenidos.

Un cálculo simple con la ley de Little muestra la magnitud: si el servidor tiene un pool de 200 hilos y cada inscripción retiene un hilo 2 s, el sistema atiende como máximo 100 inscripciones por segundo, de modo que 20.000 estudiantes tardarían al menos 200 s en ser atendidos y la mayoría esperaría mucho más de 2 s. Si la VPN falla y la llamada no tiene timeout, el tiempo de retención tiende a infinito y la capacidad tiende a cero: el sistema colapsa. Por eso, en esta arquitectura, la decisión "validar el pago de forma síncrona y con bloqueo estricto durante la inscripción" es el **punto de trade-off**: mejorar la seguridad y la consistencia empeora directamente el rendimiento y la disponibilidad, y viceversa.

Puntos de Sensibilidad y Riesgos

Tabla 3

Puntos de sensibilidad, trade-off y riesgos identificados en el SUMP

Hallazgo	Tipo	Descripción
S1. Llamada REST síncrona Java → Node por VPN	Punto de sensibilidad	El tiempo de respuesta y la disponibilidad de la inscripción dependen de la latencia de la VPN y de AWS: si AWS tarda 3 s, la inscripción tarda al menos 3 s.
S2. Granularidad y duración de los bloqueos en Oracle	Punto de sensibilidad	Bloqueos amplios o largos (p. ej., sobre la tabla de cupos) serializan a miles de estudiantes que compiten por la misma asignatura.
T1. Validación de pago síncrona + ACID estricto en el flujo de inscripción	Punto de trade-off	Mejora seguridad/consistencia (+) y empeora rendimiento/disponibilidad (−).
R1. Sin timeout ni circuit breaker	Riesgo crítico	Cada fallo de la VPN deja un hilo bloqueado indefinidamente; con 20.000 usuarios el pool se agota en segundos y el sistema colapsa a las 8:00 a. m.
R2. Sin transacción distribuida ni idempotencia entre Oracle y la BD de pagos	Riesgo crítico	Si el pago se aprueba en AWS pero la respuesta se pierde en la VPN, el monolito no registra el pago (o el estudiante reintenta y paga dos veces): explica los "pagos perdidos" y las "transacciones huérfanas".
R3. Monolito en un único servidor físico on-premise	Riesgo	Punto único de falla y sin capacidad de escalado elástico para el pico semestral.
N1. Oracle con transacciones ACID para los datos académicos	No-riesgo	Decisión adecuada para la integridad de las inscripciones; el problema es dónde y cuánto tiempo se bloquea, no ACID en sí.

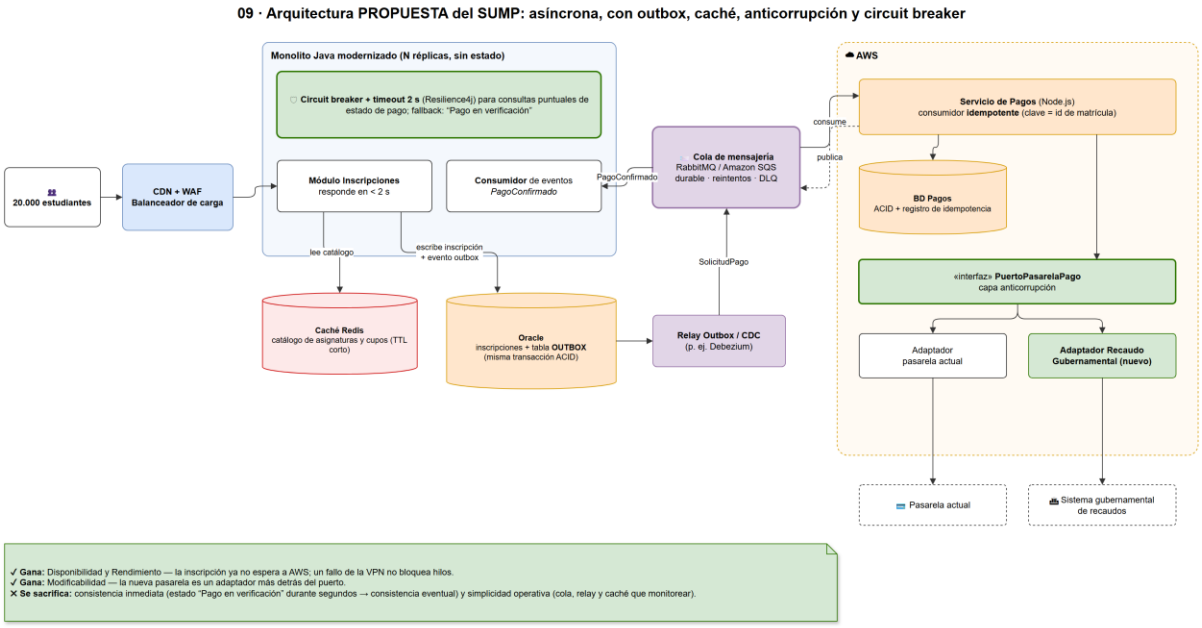
Nota. Elaboración propia según la descripción del caso SUMP.

Solución Arquitectónica Propuesta

Propongo **sacar la validación del pago del camino crítico de la inscripción** sin debilitar los controles financieros, mediante comunicación asíncrona y patrones de fiabilidad (Figura 26):

Figura 26

Arquitectura Propuesta del SUMP



Nota. Elaboración propia en diagrams.net (`diagramas/09_sump_propuesto.drawio`).

- **Cola de mensajería durable** (RabbitMQ o Amazon SQS) entre el monolito y Pagos, con reintentos y cola de mensajes muertos (DLQ). La inscripción reserva el cupo y responde en menos de 2 s con el estado "Cupo reservado — pago en verificación"; ya no espera a AWS.
- **Patrón outbox transaccional** (Richardson, 2018): en la misma transacción ACID de Oracle se guarda la inscripción y un registro "SolicitudPago" en una tabla OUTBOX; un relay (o CDC con Debezium) lo publica en la cola. Así nunca se registra una inscripción sin su mensaje, ni un mensaje sin inscripción: desaparecen las transacciones huérfanas.
- **Consumidor idempotente en Pagos** con su propia transacción ACID y las 5 validaciones y la confirmación criptográfica que exige Finanzas. La clave de idempotencia (el identificador

de matrícula) garantiza que un mensaje repetido no cobre dos veces. Al terminar, Pagos publica `PagoConfirmado` y el monolito lo consume para confirmar la matrícula.

- **Caché Redis** para lecturas masivas (catálogo de asignaturas, horarios y cupos aproximados), que son la mayoría del tráfico de las 8:00 a. m., quitando carga a Oracle.
- **Circuit breaker y timeouts** (p. ej., Resilience4j con timeout de 2 s) en cualquier llamada síncrona que permanezca, como la consulta puntual del estado de un pago, con un *fallback* "Pago en verificación" en lugar de bloquear hilos (Nygard, 2018; Fowler, 2014).
- **Escalado horizontal del monolito** (instancias sin estado detrás de un balanceador, idealmente en la nube) y una sala de espera virtual para dosificar la entrada a las 8:00 a. m.
- **Puerto `PasarelaPago` con adaptadores (capa anticorrupción)** dentro del servicio de Pagos, que además prepara el cambio exigido por el Ministerio (Módulo B).

Tabla 4

Atributos de calidad que mejoran y que se sacrifican con la solución propuesta

Atributo	Efecto	Justificación
Disponibilidad	Mejora ✅	Un fallo de la VPN o de AWS ya no bloquea hilos: los mensajes esperan en la cola.
Rendimiento	Mejora ✅	La inscripción responde en < 2 s porque no espera la validación remota; la caché absorbe las lecturas.
Seguridad / integridad financiera	Se conserva ✅	Las 5 validaciones, la firma y el ACID siguen existiendo, en la transacción local de Pagos, fuera del camino crítico.
Modificabilidad	Mejora ✅	La pasarela queda detrás de un puerto; el monolito ya no conoce detalles del proveedor.
Consistencia inmediata	Se sacrifica ⚠	Durante segundos o minutos la matrícula está "pendiente de pago" (consistencia eventual); hay que comunicarlo bien en la UI.
Simplicidad operativa y costo	Se sacrifica ⚠	Aparecen cola, relay, caché y circuit breakers que monitorear, y depurar un flujo asíncrono es más difícil.

Nota. Elaboración propia.

Módulo B: Aplicación de SAAM

Escenario del Ministerio de Educación: en exactamente 30 días todo el sistema debe cambiar su pasarela de pagos e integrarse con un nuevo sistema gubernamental de recaudos. Es un escenario **indirecto** (la arquitectura actual no lo soporta sin modificaciones), así que aplico la técnica de SAAM: recorrer la arquitectura y contar los componentes que deben modificarse (Figura 24, panel derecho).

Tabla 5

Componentes del SUMP afectados por el cambio de pasarela de pagos

Componente	¿Se modifica?	Por qué
Microservicio de Pagos (Node.js, AWS)	Sí	Contiene la integración con la pasarela: API, autenticación, formatos, estados y notificaciones (webhooks) del nuevo sistema gubernamental.
Monolito Java (controlador PaymentGateway y vistas JSP de pago)	Sí	El contrato REST síncrono entre el monolito y Node refleja los datos de la pasarela antigua (referencias, estados); el formulario de pago cambia según lo que exija el recaudo gubernamental.
Base de datos Oracle	Sí	Guarda referencias, estados de pago y datos de conciliación propios del proveedor actual; requiere migración de esquema y de datos históricos.
BD del microservicio de Pagos	Sí	Nuevos campos y estados del sistema gubernamental.
VPN / red	Probablemente	Los sistemas gubernamentales suelen exigir IP fijas, certificados y TLS mutuo: nueva configuración de red y seguridad.
Reportes y conciliación de Contabilidad	Probablemente	Los archivos de conciliación cambian de formato y de origen.

Nota. Elaboración propia.

Conteo y veredicto. Los cuatro componentes principales que menciona el enunciado se ven afectados con seguridad —monolito (incluida su interfaz de pago), microservicio, base de datos Oracle y la base de datos de pagos—, más dos probables (red y reportes): entre 4 y 6 de 6 componentes, en dos tecnologías (Java y Node.js), dos entornos (on-premise y AWS) y a través de una VPN inestable. El cambio **no queda aislado**: se propaga en cascada porque no existe ninguna abstracción que oculte la pasarela, el monolito conoce detalles del microservicio y la

base de datos central almacena datos específicos del proveedor. Además, el mismo componente (el monolito) aparece afectado en casi todos los escenarios de cambio, señal de baja separación de responsabilidades. **Veredicto: la arquitectura actual NO es altamente modificable frente a este escenario**; tiene modificabilidad baja y existe un riesgo alto de incumplir el plazo legal de 30 días. La recomendación es introducir de inmediato un contrato canónico interno de pagos y un puerto `PasarelaPago` con un adaptador por proveedor (Figura 26): el cambio actual seguiría costando, pero el siguiente afectaría a un solo componente.

Módulo C: Auditoría UX con las Heurísticas de Nielsen

Analicé el flujo del estudiante descrito en el caso e identifiqué diez violaciones a las heurísticas de Nielsen (1994b). Las 5.000 llamadas diarias al centro de ayuda son la consecuencia medible de estas fallas.

Tabla 6

Violaciones a las heurísticas de Nielsen en el flujo del SUMP y solución de diseño

#	Evento del flujo	Heurística violada	Por qué la viola	Solución de diseño (Frontend/UI)
1	Carrusel de noticias que se mueve rapidísimo en la pantalla principal	Diseño estético y minimalista	Información irrelevante para la tarea (inscribir) compite con la importante y distrae; el movimiento rápido impide leer.	Quitar el carrusel de la página de acceso o dejarlo estático (rotación ≥ 7 s, con pausa); dar protagonismo al botón de ingreso.
2	Botón “Ejecutar procedimiento de autenticación”	Relación entre el sistema y el mundo real	Usa jerga técnica en lugar del lenguaje del estudiante.	Etiqueta “Iniciar sesión”, con alto contraste (≥ 4,5:1 según WCAG) y como botón primario.
3	Botón gris claro sobre fondo blanco	Visibilidad del estado del sistema (y diseño estético y minimalista)	La acción principal no se percibe; parece deshabilitada.	Botón primario de color institucional con texto blanco y foco visible.
4	El buscador no acepta “Ingeniería de Software”, exige el código exacto ING-SOFT-402-A	Reconocer antes que recordar	Obliga a memorizar un código arbitrario en lugar de reconocer la asignatura por su nombre.	Búsqueda por nombre o código con autocompletado, tolerante a mayúsculas, tildes y guiones; lista filtrable del plan de estudios.
5	Un guion mal puesto deja la pantalla en blanco sin mensaje	Ayudar a reconocer, diagnosticar y recuperarse de errores	No explica qué pasó ni cómo corregirlo; el estudiante queda perdido.	Mensaje en lenguaje claro: “No encontramos ING-SOFT402A. ¿Quisiste decir ING-SOFT-402-A?”, conservando lo escrito.
6	“Inscribir” tarda 45 s sin cambio de cursor ni barra de carga	Visibilidad del estado del sistema	El estudiante no sabe si el sistema está trabajando, si falló o si debe esperar.	Spinner y mensaje “Inscribiendo tu asignatura…” con progreso por pasos; procesamiento asíncrono con notificación al terminar.

#	Evento del flujo	Heurística violada	Por qué la viola	Solución de diseño (Frontend/UI)
7	El estudiante hace clic 5 veces más en “Inscribir”	Prevención de errores	La interfaz permite repetir una acción no idempotente, con riesgo de inscripciones o cobros duplicados.	Deshabilitar el botón tras el primer clic y usar una clave de idempotencia en el backend.
8	Botón “Atrás” en la pantalla de pago → “Excepción fatal: NullPointerException… Transacción huérfana”	Control y libertad del usuario; ayudar a reconocer, diagnosticar y recuperarse de errores; consistencia y estándares	No hay salida de emergencia, el botón Atrás no se comporta según el estándar web y el mensaje expone detalles técnicos internos (también un riesgo de seguridad).	Botón visible “Cancelar pago y volver”; manejar la navegación hacia atrás sin perder el estado; mensaje amable: “Tu pago no se realizó. Tu inscripción sigue guardada.”
9	Olvidar la fecha de vencimiento de la tarjeta borra todo el formulario	Prevención de errores (y control y libertad del usuario)	Castiga un error menor con la pérdida de todo el trabajo; no valida antes de enviar.	Validación en línea campo por campo, marcar en rojo solo el campo faltante, conservar los demás datos y deshabilitar “Pagar” hasta completar los obligatorios.
10	5.000 llamadas diarias al centro de ayuda	Ayuda y documentación	No hay ayuda contextual enfocada en la tarea; el soporte telefónico es la única vía.	Ayuda contextual (íconos “?”), guía paso a paso de inscripción, preguntas frecuentes buscables y chat.

Nota. Elaboración propia a partir del flujo descrito en el caso SUMP.

Cuestionario del Taller

Pregunta 1. Rendimiento (Latencia)

Definiría el requisito así: **durante un lanzamiento de alta demanda, con 10.000 usuarios concurrentes (≈ 5.000 solicitudes por segundo), la consulta de disponibilidad de asientos debe responder con un percentil 95 ≤ 200 ms y un percentil 99 ≤ 500 ms, medidos en el API Gateway, con una tasa de error < 0,1 %**. Uso percentiles y no promedios porque el promedio oculta a los usuarios que esperan varios segundos. El valor de 200 ms deja margen para la red del usuario y para el renderizado, de modo que la percepción total quede por debajo de 1 s, que es el umbral en el que el usuario siente que el sistema responde sin interrumpir su flujo. Para cumplirlo, esa consulta se sirve desde una caché (Redis) con datos de 1 a 2 s de antigüedad, nunca desde la base de datos transaccional.

Pregunta 2. Escalabilidad (Horizontal)

Pasar de 1.000 a 100.000 usuarios concurrentes (100 veces más) exige escalar horizontalmente, no comprar un servidor más grande. Implementaría:

- **Microservicios sin estado** detrás de un **balanceador de carga**, con **autoescalado** (p. ej., Kubernetes HPA con CPU > 60 % o longitud de cola como métrica) y escalado preventivo antes de la hora anunciada de venta.
- **CDN** para contenido estático y **caché** para el catálogo y la disponibilidad, que representan la mayoría de las lecturas.
- **Sala de espera virtual** (cola de admisión) que deja entrar usuarios a la compra a un ritmo que el sistema puede sostener, con posición visible en la fila.
- **Procesamiento asíncrono** con un bus de eventos (Kafka, cuyo número de particiones define el paralelismo máximo de consumidores).

- **Escalado de datos**: réplicas de lectura y particionamiento (*sharding*) del inventario por `eventoId`, para que un festival no compita con otros eventos.
- **Pruebas de carga** (k6 o JMeter) al 120 % de la carga esperada antes del evento.

Pregunta 3. Fiabilidad (Tolerancia a Fallos)

Combinaría varios mecanismos, porque ninguno basta por sí solo:

- **Clave de idempotencia**: el cliente genera un UUID por intento de pago (`Idempotency-Key`) y lo envía también a la pasarela; si el usuario o el sistema reintentan, el servidor reconoce la clave y devuelve el resultado original en lugar de cobrar de nuevo. Esto evita la **duplicación**.
- **Máquina de estados persistente de la orden** (PENDIENTE_PAGO → EN_VERIFICACIÓN → PAGADA/RECHAZADA) y la reserva de asientos mantenida 10 minutos: si Pagos cae en el paso 3, la orden no se pierde; queda en verificación. Esto evita la **pérdida**.
- **Reintentos con retroceso exponencial y jitter** (p. ej., 3 intentos a 1, 2 y 4 s) solo para errores transitorios.
- **Circuit breaker**: si Pagos falla repetidamente (p. ej., 50 % de errores en 20 solicitudes), el circuito se abre, se deja de insistir y se responde rápido “Estamos verificando tu pago”; tras un tiempo prueba en estado semiabierto.
- **Outbox + consumidor idempotente y conciliación**: el resultado del pago se publica de forma fiable y un proceso de conciliación consulta a la pasarela por la clave de idempotencia las órdenes que quedaron en verificación.

Pregunta 4. Seguridad (Autorización)

Implementaría RBAC con verificación de propiedad, aplicando el principio de "denegar por defecto":

- Roles y permisos: `ORGANIZADOR` → `ventas:leer:propias`; `ADMIN` → `ventas:leer:todas`.
- Autenticación con OAuth 2.0/OpenID Connect: el token JWT incluye el identificador del usuario (`sub`) y sus roles; el API Gateway valida la firma y la expiración.
- Autorización en el servicio, no solo en la interfaz: si el rol es ADMIN, la consulta no se filtra; si es ORGANIZADOR, se filtra siempre por propietario (`WHERE organizador_id = :sub`) y una solicitud a un evento ajeno devuelve 403 (o 404 para no revelar que existe).
- Política centralizada (middleware o motor de políticas), registro de auditoría de los accesos y pruebas automatizadas que verifiquen que un organizador nunca ve ventas ajenas.

Pregunta 5. Seguridad (Integridad de Datos)

La medida específica es emitir cada entrada con un **identificador único no adivinable (UUID) y un código QR firmado digitalmente** por EventFlow (firma HMAC-SHA256 o Ed25519 sobre el ID de la entrada, el evento y la fecha). En la puerta, el lector verifica la firma —un QR falsificado no tiene una firma válida— y ejecuta una **transición atómica de un solo uso** en la base de datos (`UPDATE entradas SET estado='USADA' WHERE id=:id AND estado='VENDIDA'`): si actualiza 0 filas, la entrada ya se usó y se rechaza. Para la reventa, la transferencia oficial dentro de la plataforma revoca el QR anterior y emite uno nuevo; y un QR dinámico que rota cada 30 s en la app evita que circulen capturas de pantalla.

Pregunta 6. Usabilidad (Eficiencia)

Propondría una **carga masiva por plantilla CSV/Excel** con vista previa y validación por fila ("Fila 17: la fecha está en el pasado"), de modo que el promotor corrija solo las filas con error y confirme las 50 de una vez; además, **duplicar un evento existente** como plantilla y una API para integrarse con su propio sistema. Esto aplica la heurística de *flexibilidad y eficiencia de uso*: aceleradores para usuarios expertos que no estorban al novato. La meta medible sería pasar de unas 4 horas semanales (≈ 5 minutos por evento) a menos de 10 minutos.

Pregunta 7. Mantenibilidad (Modularidad)

Con una **arquitectura hexagonal (puertos y adaptadores)** dentro del Servicio-Pagos: el dominio define una interfaz propia `PasarelaPago` (`autorizar`, `capturar`, `reembolsar`, `consultarEstado`) con sus propios modelos y estados, y cada proveedor es un adaptador (`StripeAdapter`, `PayUAdapter`) que traduce entre ese modelo y la API externa —una capa anticorrupción—. El adaptador activo se elige por configuración o *feature flag* (patrón Strategy con inyección de dependencias), y las pruebas de contrato garantizan que ambos se comportan igual. Así, el cambio de Stripe a PayU toca un solo componente; Pedidos y Entradas siguen recibiendo el mismo evento `PagoAprobado` y ni se enteran.

Pregunta 8. Rendimiento (Concurrencia)

El desafío es una **condición de carrera**: los dos usuarios leen "F-12 disponible" al mismo tiempo y ambos escriben "vendido"; sin control, se produce una sobreventa (la segunda escritura pisa a la primera). Las opciones son:

- **Bloqueo pesimista** (`SELECT … FOR UPDATE`): el primero bloquea la fila y el segundo espera. Es seguro, pero si el bloqueo se mantiene mientras el usuario paga, genera contención y agota conexiones.

- **Bloqueo optimista / actualización condicional**: `UPDATE asientos SET estado='RESERVADO', reservado_por=:u, version=version+1 WHERE id='F-12' AND estado='DISPONIBLE' AND version=:v`. Si afecta 0 filas, el otro usuario ganó y se le informa de inmediato.

Elegiría el **optimista con reserva temporal con expiración (TTL de 10 minutos)**: la sección crítica dura milisegundos, no se retienen bloqueos durante el pago, y como la gran mayoría de las solicitudes no compiten por el mismo asiento, los conflictos son raros. El pesimista lo reservaría solo para secciones críticas muy cortas, nunca a lo largo de una interacción con el usuario.

Pregunta 9. Disponibilidad (99,99 %)

"Cuatro nueves" equivalen a unos 52,6 minutos de caída al año (≈ 4,3 minutos al mes). EventFlow lo necesita porque sus ingresos se concentran en minutos: una caída de 10 minutos justo cuando abre la venta de un concierto masivo puede significar perder la mayor parte de la recaudación de ese evento, empujar a los fans hacia la reventa, dañar la reputación y violar el acuerdo de nivel de servicio (SLA) con el organizador. El componente más crítico es el **camino de compra, y en particular el Servicio de Inventario de Entradas (reservas) con su base de datos**, porque tiene estado y es el Dominio Core: el API Gateway y los servicios sin estado se replican fácilmente, pero el inventario requiere réplicas sincronizadas en varias zonas, conmutación por error automática y respaldo continuo.

Pregunta 10. Trade-off (Rendimiento vs. Consistencia)

Elegiría un **enfoque híbrido**: **consistencia eventual para mostrar** el conteo ("Quedan aprox. 120 entradas", servido desde caché y actualizado cada 1–2 s) y **consistencia fuerte en el momento de reservar** (la actualización condicional atómica de la pregunta 8). Durante un pico,

las lecturas del contador superan a las compras en una proporción de cien a uno o más; si cada lectura fuera a la base de datos principal para ser 100 % exacta, esa base se convertiría en el cuello de botella y el sistema entero se volvería lento. Un contador desactualizado unos segundos es una inexactitud tolerable; una sobreventa no lo es, y esa garantía la da la escritura, no la lectura.

Pregunta 11. Stakeholders Clave para la Sesión ATAM

Tabla 7

Stakeholders clave para el ATAM de EventFlow y su impulsor de negocio

Stakeholder	Impulsor de negocio (business driver)
Product Owner / Director comercial	Maximizar las ventas y la conversión en los lanzamientos: ningún fan debe irse a la competencia por lentitud o caídas.
Director financiero (Finanzas)	Cero pagos perdidos o duplicados y cumplimiento PCI DSS: cada peso debe conciliarse.
Líder técnico de operaciones (SRE/DevOps)	Disponibilidad de 99,99 % con escalado elástico a un costo de infraestructura controlado.

Nota. Si hubiera una cuarta silla, invitaría a un representante de los organizadores, cuyo impulsor es la visibilidad de sus ventas en tiempo real.

Pregunta 12. Atributos con Mayor Prioridad en el Árbol de Utilidad

Elegiría **escalabilidad (rendimiento bajo carga pico)** y **fiabilidad (integridad transaccional del inventario y los pagos)** como (A,A). Escalabilidad, porque el modelo de negocio depende de vender en el primer minuto de un lanzamiento y absorber picos de 10 a 100 veces la carga normal es técnicamente difícil. Fiabilidad, porque vender un asiento dos veces o cobrar sin entregar la entrada destruye la confianza y genera pérdidas, y garantizarlo en un sistema distribuido (concurrencia, idempotencia, sagas) es la parte más difícil del diseño. La seguridad tiene importancia alta, pero dificultad media, porque buena parte se delega (tokenización en la pasarela, proveedor de identidad), así que la ubico como (A,M).

Pregunta 13. Escenario de Calidad de Rendimiento (Estímulo–Respuesta–Medida)

Tabla 8

Escenario de calidad de rendimiento para el lanzamiento de un evento masivo

Parte del escenario	Valor
Fuente del estímulo	10.000 fans (anónimos y autenticados) desde la web y la app móvil.
Estímulo	Consultan la disponibilidad y solicitan reservar entradas durante el primer minuto tras la apertura de la venta (pico ≈ 5.000 solicitudes/s).
Entorno	Operación en pico de un lanzamiento masivo, con autoescalado activo y una zona de disponibilidad degradada.
Artefacto	API Gateway, Servicio-Eventos (disponibilidad) y Servicio-Entradas (reservas).
Respuesta	El sistema muestra la disponibilidad y confirma o rechaza cada reserva sin errores ni sobreventa; los usuarios excedentes pasan a la sala de espera virtual con su posición visible.
Medida de la respuesta	p95 ≤ 200 ms en la consulta de disponibilidad; p95 ≤ 2 s en la confirmación de la reserva; tasa de error < 0,5 %; 0 asientos vendidos dos veces.

Nota. Formato de escenario de atributo de calidad de Bass et al. (2021).

Pregunta 14. Mapeo de la Decisión de Microservicios al Escenario de Escalabilidad

Le pediría que recorra el escenario (A,A) “10.000 usuarios en 1 minuto” sobre la arquitectura, paso a paso, y que demuestre para cada componente cómo responde:

- **¿Qué componentes reciben la carga** y en qué orden (Gateway → Servicio de Asientos → Pagos)? ¿Cuál es el cuello de botella esperado?
- **¿Qué táctica de escalabilidad usa cada uno** (instancias sin estado, réplicas mínimas y máximas, regla de autoescalado, caché, cola) y con qué evidencia de capacidad: “una réplica atiende 300 solicitudes/s; para 5.000 solicitudes/s necesito 17 réplicas más un 30 % de margen”.
- **¿Dónde vive el estado** y cómo escala: ¿la base de datos de Asientos está particionada? ¿cuántas particiones tiene el bus, que limitan el número de consumidores en paralelo?
- **¿Qué pasa con las dependencias que no escalan**: el límite de solicitudes por segundo de la pasarela de pagos o del proveedor de autenticación?

- **Resultados de pruebas de carga** que respalden la respuesta y la medida del escenario.

El entregable sería una tabla "decisión → táctica → respuesta al escenario → puntos de sensibilidad y riesgos", que es exactamente el análisis de enfoques arquitectónicos de ATAM.

Pregunta 15. Riesgo de Bloquear el Asiento en la Base de Datos al Seleccionarlo

Identificaría inmediatamente un **riesgo de contención de bloqueos**. Si el bloqueo se toma cuando el usuario selecciona el asiento, se mantiene durante todo el tiempo que tarda en decidir y pagar (minutos, no milisegundos). Con miles de usuarios, las transacciones y conexiones quedan retenidas esperando, el pool de conexiones se agota, aparecen *deadlocks* y tiempos de espera, el rendimiento colapsa justo en el pico y los asientos de usuarios que abandonan la página quedan bloqueados. Es, además, un punto de sensibilidad para el rendimiento. La alternativa es una reserva lógica con expiración (estado RESERVADO con TTL) y actualización condicional, sin bloqueos de base de datos de larga duración.

Pregunta 16. Trade-off del Cifrado de Extremo a Extremo

Se crea un punto de trade-off **Seguridad (+) frente a Rendimiento (−)**: cifrar y descifrar todo con alto consumo de CPU aumenta la latencia de cada solicitud y reduce el *throughput*, lo que también afecta la **escalabilidad** (cada instancia atiende menos usuarios) y el **costo** (más instancias para la misma carga); de forma secundaria, afecta la mantenibilidad (gestión de claves) y la funcionalidad (no se puede buscar ni indexar sobre datos cifrados). Lo mitigaría cifrando de forma selectiva: TLS 1.3 con aceleración por hardware en el transporte, cifrado a nivel de campo solo para datos personales sensibles, cifrado por sobres con un servicio KMS y tokenización de la tarjeta, que es más segura y más barata que cifrarla.

Pregunta 17. Punto de Sensibilidad Crítico para la Mantenibilidad

La decisión más sensible es **la definición de los límites de los servicios y de sus contratos** (las APIs y, sobre todo, el esquema de los eventos del bus). Si los límites están mal trazados —por ejemplo, si Pedidos y Entradas comparten una base de datos o si un evento `PagoAprobado` expone la estructura interna de Pagos—, cualquier cambio en un servicio obliga a modificar y desplegar a los demás de forma coordinada: un monolito distribuido. Por eso lo trataría como un punto de sensibilidad explícito: una base de datos por servicio, esquemas de eventos versionados y compatibles hacia atrás, y pruebas de contrato.

Pregunta 18. Preguntas al Arquitecto sobre la Caída de la Pasarela de Pagos

- ¿Qué timeout se configura para la llamada a Stripe y qué ocurre con el hilo y con la orden cuando se agota?
- ¿Se reintenta? ¿Cuántas veces, con qué retroceso, y se envía la misma clave de idempotencia para no cobrar dos veces?
- Si Stripe cobró pero la respuesta se perdió, ¿cómo se entera EventFlow (webhooks, consulta de estado, conciliación)?
- ¿En qué estado queda la orden y qué ve el usuario? ¿Cuánto tiempo se mantiene la reserva de sus asientos?
- ¿Hay circuit breaker? ¿Con qué umbrales se abre y qué *fallback* ofrece (cola de pagos pendientes, pasarela alternativa)?
- ¿La caída de Stripe afecta a otras funciones (catálogo, login) o está aislada mediante *bulkheads*?
- ¿Cómo se mantiene la consistencia entre Pedidos y Pagos (saga, acciones de compensación)?

- ¿Qué monitoreo y alertas detectan la caída y en cuántos minutos se notifica al equipo de guardia?
- ¿El SLA de Stripe es compatible con nuestro objetivo de 99,99 %? ¿Se ha probado este escenario con ingeniería del caos?

Pregunta 19. Hallazgo Más Valioso para el Gerente de Proyecto

Para un gerente de proyecto, el hallazgo más valioso es **un riesgo identificado** (idealmente agrupado en temas de riesgo y ligado a un impulsor de negocio). La razón es que el riesgo es **accionable** y se traduce directamente en las variables que el gerente controla —alcance, costo, cronograma y calidad—: se convierte en historias de mitigación en el backlog, en presupuesto, en un plan de contingencia y en una decisión informada antes de que el problema ocurra en producción, que es cuando más cuesta. Un no-riesgo da tranquilidad, pero no exige acción, y un punto de trade-off documentado es muy útil para negociar con los stakeholders, pero su valor se materializa cuando se analiza qué riesgo genera. Por eso el riesgo es lo que más protege el éxito del proyecto (Clements et al., 2002).

Conclusiones

- Los atributos de calidad, expresados como escenarios medibles, son los verdaderos impulsores de la arquitectura: el NFR de 10.000 usuarios por minuto basta para descartar el monolito en capas como solución final de EventFlow.
- Los microservicios con bus de eventos resuelven la escalabilidad y el aislamiento de fallos mediante escalado horizontal selectivo, alta cohesión y bajo acoplamiento, pero no son "fáciles": trasladan la complejidad a la red, a los datos distribuidos y a la operación.
- La invocación implícita (publicar/suscribir) desacopla a los productores de los consumidores; el propio pipeline del taller (`on: push`) es un ejemplo de ese principio.
- El ciclo DevOps simulado sobre los archivos del taller mostró cómo la automatización (GitHub Actions), el análisis estático (SonarCloud) y la infraestructura como código (Terraform) hacen los procesos repetibles, fiables y auditables, y cómo los malos olores del `index.html` se convierten en deuda técnica cuantificable.
- Las heurísticas de Nielsen permiten detectar problemas de usabilidad concretos y baratos de corregir; "agradar visualmente" y "facilitar la navegación" son objetivos, no heurísticas.
- En el SUMP, la llamada síncrona sin timeout por una VPN inestable es a la vez punto de sensibilidad, punto de trade-off y riesgo crítico; desacoplarla con una cola, outbox, idempotencia, caché y circuit breaker mejora la disponibilidad y el rendimiento sin sacrificar la seguridad financiera, a cambio de consistencia eventual y mayor complejidad operativa.
- SAAM demostró que el SUMP tiene baja modificabilidad frente al cambio de pasarela (de 4 a 6 componentes afectados), mientras que un diseño con puertos y adaptadores reduce ese impacto a un solo componente.

Referencias

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.

Clements, P., Kazman, R., & Klein, M. (2002). *Evaluating software architectures: Methods and case studies*. Addison-Wesley.

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2014). *CircuitBreaker*. martinfowler.com. https://martinfowler.com/bliki/CircuitBreaker.html

Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2nd ed.). Addison-Wesley.

Garlan, D., & Shaw, M. (1994). *An introduction to software architecture* (Technical Report CMU-CS-94-166). Carnegie Mellon University.

GitHub. (s.f.). *GitHub Actions documentation*. https://docs.github.com/en/actions

HashiCorp. (s.f.). *Terraform documentation*. https://developer.hashicorp.com/terraform/docs

Kazman, R., Bass, L., Abowd, G., & Webb, M. (1994). SAAM: A method for analyzing the properties of software architectures. En *Proceedings of the 16th International Conference on Software Engineering* (pp. 81–90). IEEE.

Kazman, R., Klein, M., & Clements, P. (2000). *ATAM: Method for architecture evaluation* (Technical Report CMU/SEI-2000-TR-004). Software Engineering Institute, Carnegie Mellon University.

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations*. IT Revolution Press.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

Nielsen, J. (1994a). Enhancing the explanatory power of usability heuristics. En *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 152–158). ACM.

Nielsen, J. (1994b). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.

Richardson, C. (2018). *Microservices patterns*. Manning.

SonarSource. (s.f.). *SonarQube Cloud documentation* (antes SonarCloud). https://docs.sonarsource.com/sonarqube-cloud/